

ELAN

USER MANUAL

V3.6 (2003), rebuilt 2026
ELAN V3.6

Peter Borovanský
Horatiu Cirstea
Hubert Dubois
Claude Kirchner
Hélène Kirchner
Pierre-Etienne Moreau
Quang-Huy Nguyen
Christophe Ringeissen
Marian Vittek

ELAN: USER MANUAL

Authors: Peter Borovanský, Horatiu Cirstea, Hubert Dubois, Claude Kirchner, Hélène Kirchner, Pierre-Etienne Moreau, Quang-Huy Nguyen, Christophe Ringeissen, Marian Vittek

LORIA: CNRS, INPL, INRIA, UHP, Université de Nancy 2
Campus scientifique
615, rue du Jardin Botanique
BP101
54602 Villers-lès-Nancy CEDEX
FRANCE
E-mail: elan@loria.fr

Copyright ©1996 – 2002 Peter Borovanský, Horatiu Cirstea, Hubert Dubois, Claude Kirchner, Hélène Kirchner, Pierre-Etienne Moreau, Quang-Huy Nguyen, Christophe Ringeissen and Marian Vittek.

Permission is granted to make and distribute verbatim copies of this book provided the copyright notice and this permission are preserved on all copies.

elan n. **1.** Enthusiastic vigor and liveness. **2.** Style; flair. [Fr < OFr. *eslan*, rush < *eslancer*, to throw out: es-, out (< Lat. ex-) + *lancer*, to throw (< LLat. *lanceare*, to throw a lance < Lat. *lancea*, lance).]

The American Heritage Dictionary

- 1. ÉLAN.** *n.m.* (vers 1420 selon BLOCH ; de *élancer*.
|| **1** Mouvement pour s'élancer. *Calculer son élan* (Cf. Calculer, cit. 7). ...
|| **2** *Par anal.* Mouvement par lequel la voix reprend, s'élance. ...
|| **3** *Fig.* Mouvement ardent, subit qu'un vif sentiment inspire. V. **Mouvement; ardent, impulsion, poussée.** *Brusque élan. Élan impétueux. Élans de jeunesse, de passion.* ...
- 2. ÉLAN.** *n.m.* (XVII^e s.; *hellent* au XV^e s., *ellend* au XVI^e; haut allem. *elend*, du baltique *elnis*). Mammifère artiodactyle (*Cervidés*), des pays du Nord. *Élan du Canada*. V. **Original.**

LE ROBERT, extraits.

Contents

1	A short introduction to ELAN	7
1.1	What could you do in ELAN?	7
1.2	A very simple example	8
1.3	A more generic example	9
1.4	An extended example	11
2	How to use ELAN	15
2.1	Using ELAN	15
2.1.1	How to run the ELAN interpreter	15
2.1.2	Top level interpreter	20
2.1.3	How to run the ELAN compiler	25
2.2	Keep in touch	27
3	ELAN: the language	29
3.1	Lexicographical conventions	29
3.1.1	Separators	29
3.1.2	Lexical unities	29
3.1.3	Comments	30
3.2	Element modules	30
3.3	Definition of signatures	31
3.3.1	Sort declarations	31
3.3.2	Operator declarations	32
3.3.3	Function declaration options	34
3.3.4	Strategy declaration	35
3.4	Definition of rules	36
3.4.1	Rule syntax	36
3.4.2	The where construction	38
3.4.3	Choose-Try	39
3.4.4	Conditions	40
3.4.5	Labels visibility	40
3.5	Definition of strategies	40
3.5.1	Elementary strategies syntax	41
3.5.2	Defined strategies	47
3.6	Evaluation mechanism	48
3.6.1	Rewrite rules on terms	48
3.6.2	Rewrite rules on strategies	52
3.7	Modules	55
3.7.1	Visibility rules	55
3.7.2	Built-in modules	56

3.7.3	Parameterised modules	56
3.7.4	LGI modules	57
3.8	Pre-processing	59
3.8.1	Simple duplication	59
3.8.2	Duplication with argument	60
3.8.3	Enumeration using FOR EACH	60
3.9	Differences with previous version of the language	61
4	ELAN: the system	63
4.1	Global architecture	63
4.2	The preprocessor	63
4.3	The parser	63
4.4	The interpreter	64
4.5	The compiler	64
4.5.1	Non-linear rules	65
4.5.2	Built-ins	65
4.5.3	Input/Output facilities	65
4.6	The REF format	66
5	The standard library and the built-ins	69
5.1	The ELAN standard library	69
5.1.1	common	69
5.1.2	ref	70
5.1.3	noquote	70
5.1.4	strategy	70
5.2	The built-ins	70
5.2.1	Booleans	71
5.2.2	Numbers	71
5.2.3	Identifiers	73
5.2.4	Elementary term computations	74
5.2.5	Input/Output	75
5.2.6	Strategies	77
5.2.7	REF Format	81
5.3	Calling external programs	82
6	Contributed works	87
6.1	Description of the ELAN contributions	87
6.2	ELAN's annotated bibliography	88

Foreword

This manual presents the version V3.6 of the ELAN language and of its environment.

This is mainly an update of version V3.4 with bug corrections with several new features offered by the compiler such that the Input/Output facilities, the new builtins (array, hashing tables) and the possibility to trace the normalisation process in ELAN programs. The ELAN compiler compiles now all possible patterns that contain AC symbols. No program transformation is required for the user. Furthermore, from this version we distribute the sources of the system. The compilation/installation process follows the same procedure as other GNU softwares.

This is the user manual of ELAN version V3.6

We will be pleased to know any inaccuracy, error or typos.

Your comments on any part of the language, the system or this manual are welcome.

`elan@loria.fr`

Chapter 1

A short introduction to ELAN

This chapter presents in a top-down approach the ELAN main features. If you are beginning to use ELAN, you should certainly have a look at this chapter first. Complementarily Chapter 3 presents a bottom-up complete description of the language.

For a gently introduction to ELAN we recommend to read [BKK⁺98b] or to access to <http://elan.loria.fr>.

1.1 What could you do in ELAN?

Relying on the premiss that inferences rules can be quite conveniently described by rewrite rules, we started in the early nineties to design and implement a language in which inference systems can be represented in a natural way, and executed reasonably efficiently.

This leads us quickly to formalise such a language using the rewriting logic introduced by J. Meseguer [Mes92] and to see ELAN as a logical framework where the frame logic is rewriting logic extended with the fundamental notion of strategies [Vit94, KKV95]. A rewrite theory and an associated strategy is called a *computational system*.

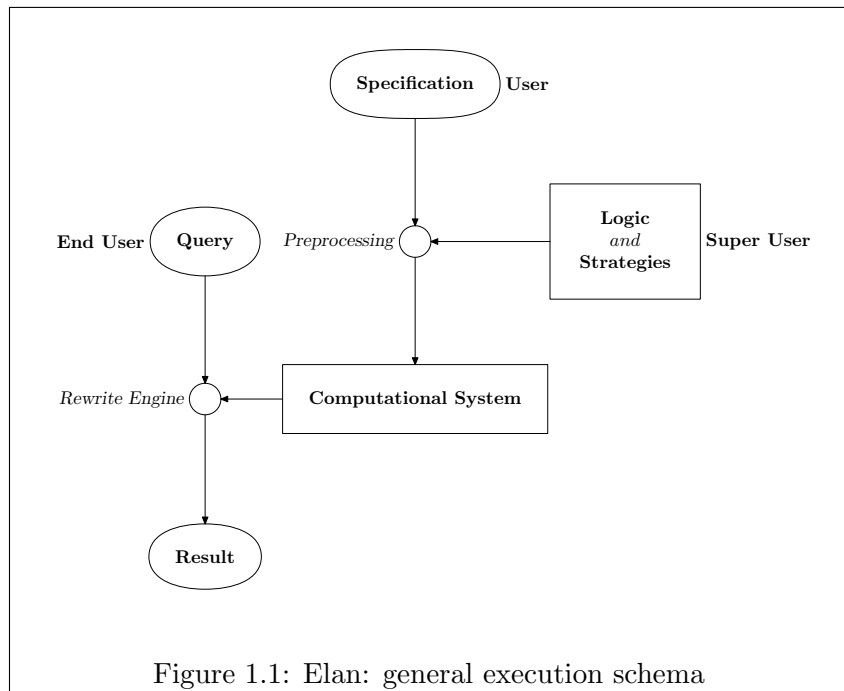
In ELAN, a logic can be expressed by specifying its syntax, its inference rules and a description of how these rules should be applied. The syntax of the logic can be described using mixfix operators as in the OBJ language [GKK⁺87] or SDF [HHKR89]. The inference rules of the logic are described by conditional rewrite rules with **where** assignments allowing to introduce variables local to a rule.

From this description of the logic and a theory written in this logic, we infer automatically a computational system consisting of a rewriting theory plus strategies.

Since we wanted ELAN to be modular, with a syntactic level well-adapted to the user's needs, the language is designed in such a way that three programming levels are possible.

- First the design of a logic is done by the so-called *super-user*, with the help of a powerful preprocessor that expands specific macros into ELAN constructions.
- Such a logic can be used by the (standard) *user* in order to write a specification.
- Finally, the *end-user* can evaluate queries in the specification, following the semantics described by the logic.

This corresponds to the general diagram given in Figure 1.1. The query is interactively given by the end-user at the ELAN prompt level.



1.2 A very simple example

Let us fully detail a complete example. The next module illustrates what the programmer has to write in order to describe the derivative operation on simple polynomials. The first part contains:

- the module name: `poly1`
- modules you want to import: `int` (the integer library module)
- sort declaration: `variable` and `poly`
- operators definitions: constructors and functions declarations

The second part contains the computation definition. Given a query, **ELAN** repeatedly normalises the term using unlabelled rules. This is done in order to perform functional evaluation and thus it is recommended for the user to provide a confluent and terminating unlabelled rewrite system to ensure termination and unicity of the result. This normalisation process is built in the evaluation mechanism and consists in a leftmost innermost normalisation. This yields always a *single* result.

You can control the **ELAN** construction of terms by giving parsing annotations like associativity or priority to operators.

```
module poly1
import global int;
end
sort variable poly;
end
operators global
  X      :      variable;
  @      : ( variable )    poly;
  @      : ( int )        poly;
```

```

@ + @          : ( poly poly )      poly  assocRight pri 1;
@ * @          : ( poly poly )      poly  assocRight pri 2;
deriv(@)       : ( poly )           poly;
end

rules for poly
  p1, p2 : poly;
  n : int;
global
  [] deriv(X)          => 1          end
  [] deriv(n)          => 0          end
  [] deriv(p1+p2)      => deriv(p1)+deriv(p2)  end
  [] deriv(p1*p2)      => deriv(p1)*p2 + p1*deriv(p2)  end
end
end

```

The top level of the logic description given in the following module describes the way to run the system. The logic declaration just introduces the sorts of interest (query and result) and defines `.eln` modules to be imported: this is the “top level”.

```

LPL poly1 description
  query   of sort      poly
  result of sort      poly
  import   poly1
  start with          ()query
end

```

Once defined, we can use the two previous modules by calling the interpreter as detailed in section 2.1.1.

```

% elan poly1.lgi
enter query term finished by the key word 'end':
  deriv(X) end

[] start with term: deriv(X)
[] result term:      1

enter query term finished by the key word 'end':
  deriv(3*X*X + 2*X + 7) end

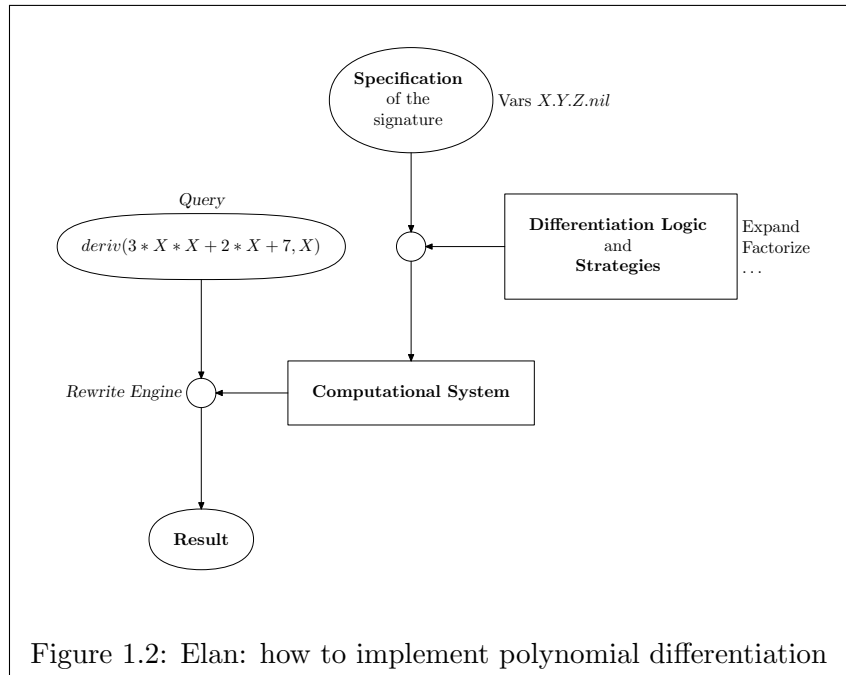
[] start with term: deriv(3*X*X+2*X+7)
[] result term:      0*X*X+3*1*X+X*1+0*X+2*1+0

```

1.3 A more generic example

The next example, introduces more ELAN features. It consists of the specification of elementary polynomials built over a finite set of variables, and the derivative functions to compute the derivated form with respect to a given variable. Tasks are divided as follows:

1. the super-user describes in a generic way the derivative and factorisation inferences, i.e. a logic for polynomial differentiation,
2. the user gives the specification of an algebra in which (s)he wants to derivate polynomials; in this case, this is quite simple since it amounts to specify the variables of the considered polynomial algebra,
3. the end-user gives a differentiation problem.



The diagram in Figure 1.1 thus instantiates as described in Figure 1.2.

The description of the logic and of the specification is done in the ELAN syntax described in the Chapter 3 and it can be further extended by the super-user. These descriptions are assumed to be done in files with specific extensions:

1. `.lgi` for the top level logic description, this file is written by the ELAN super-user,
2. `.eln` for a module used in a logic description, this file is also written by the ELAN super-user,
3. `.spc` for a specification, i.e. a program written in the defined logic by an ELAN user.

The `.eln` module of our example is the following:

```

module poly2[Vars]
import global int Vars eq[variable] identifier list[identifier];
end
sort variable poly;
end
operators global
  FOR EACH Id:identifier SUCH THAT Id:=(listExtract) elem(Vars) :
  { Id : variable; }

  @          : ( variable ) poly;
  @          : ( int ) poly;
  @ + @      : ( poly poly ) poly  assocRight pri 1 (AC);
  @ * @      : ( poly poly ) poly  assocRight pri 2 (AC);
  deriv(@,@) : ( poly variable ) poly;
end

rules for poly
  p, p1, p2 : poly;
  x, y : variable;
  n : int;
global
  [] 0+p      => p      end

```

```

[] 0*p          => 0                      end
[] 1*p          => p                      end
[] deriv(x,x)   => 1                      end
[] deriv(y,x)   => 0    if x != y        end
[] deriv(n,x)   => 0                      end
[] deriv(p1+p2,x) => deriv(p1,x)+deriv(p2,x) end
[] deriv(p1*p2,x) => deriv(p1,x)*p2 + p1*deriv(p2,x) end
end
end

```

Then, the logic declaration describes the way to parse the specification file that contains a finite list of variables.

```

LPL poly2 description
specification description
  part Vars          of sort list[identifier]
                      import identifier list[identifier]
end

query              of sort      poly
  result of sort   poly
  import          poly2[Vars]
  start with () query
end

```

To instantiate the *Logic* and build the *Computational System*, you have to give a specification. An example of such specification is:

```

specification someVariables
  Vars          X.Y.Z.nil
end

```

It contains a list of variables that is read and transformed into a rewrite rule: `Vars => X.Y.Z.nil`

The `FOR EACH` preprocessor construction used in the `poly2.eln` module performs a textual replacement that extracts identifiers `X`, `Y`, `Z` from the list `X.Y.Z.nil` and declare them of sort `variable`.

You can notice that operators `*` and `+` are now declared as associative and commutative, which is helpful to implement some simplification rules. One rule contains a conditional part.

Now let us call ELAN with the specification `someVariables.spc`:

```

% elan poly2 someVariables.spc
enter query term finished by the key word 'end':
  deriv(3*X*X + 2*X + 7 , X) end

[] start with term: deriv(3*X*X+2*X+1,X)
[] result term:      X+X*3+2

```

The result has been simplified in a better way, but it is still difficult to read due to the lack of parenthesis.

1.4 An extended example

This last example shows how to play with labelled rules and strategies.

Unlabelled rules are applied repeatedly, at any position, to normalize the term. A labelled rule is applied only when a strategy calls it. In this case, the rule is applied at top position

on the term. In the next module, you can recognize labelled rules: names are given between brackets.

You can define strategies in the same way you define rules and use them in local affectation parts introduced by the key word **where**. When a rule is applied, before constructing the right-hand-side, conditional parts are evaluated and variables involved in local affectation parts are instantiated by the result of the application of a strategy on a term.

```

module poly3[Vars]
import global int Vars eq[variable] identifier list[identifier];
end
sort variable poly;
end
operators global
  FOR EACH Id:identifier SUCH THAT Id:=(listExtract) elem(Vars) :
    { Id : variable; }
    @                : ( variable ) poly;
    @                : ( int ) poly;
    @ + @            : ( poly poly ) poly  assocRight pri 1 (AC);
    (@ + @)          : ( poly poly ) poly  alias @ + @:;
    @ * @            : ( poly poly ) poly  assocRight pri 2 (AC);
    (@ * @)          : ( poly poly ) poly  alias @ * @:;
    deriv(@,@)       : ( poly variable ) poly;
end

stratop global
  last_simplify : <poly>      bs;
  simplify      : <poly>      bs;
end

rules for poly
  P, p1, p2, p3, p4, p5      : poly;
  e1, e2, e3, e4             : poly;
  x, y                       : variable;
  n, n1, n2, n3              : int;
global
[] P+0                       => P                                end
[] P*0                       => 0                                end
[] P*1                       => P                                end
[] n1*n2                     => n where n:=( ) n1*n2            end
[] deriv(x,x)                => 1                                end
[] deriv(y,x)                => 0    if x != y                    end
[] deriv(n,x)                => 0                                end
[] deriv(p1+p2,x)            => p3
    where p3:=(simplify) deriv(p1,x)+deriv(p2,x)
end
[] deriv(p1*p2,x)            => p5
    where p4:=(simplify) p1*deriv(p2,x)
    where p3:=(simplify) deriv(p1,x)*p2
    where p5:=(simplify) p3+p4
end
local
[factorize] P + p1+p1         => P + 2*p1                        end

[factorize] P + n1*p1 + n2*p1 => P + n3*p1
    where n3:=( )n1+n2
end

[factorize] P + (p1*p2) + (p1*p3) => P + p4
    where p4:=(simplify) p1*(p2+p3)
end

[expand] P + (p1+p2)*(p3+p4) => P+e1+e2+e3+e4
    where e4:=(simplify) p2*p4
    where e3:=(simplify) p2*p3
    where e2:=(simplify) p1*p4
    where e1:=(simplify) p1*p3
end
[expand] P + (p1+p2)*p3 => P + e1+e2
    where e2:=(simplify) p2*p3
    where e1:=(simplify) p1*p3
end
end

```

```
strategies for poly
implicit
  [] last_simplify => repeat*(dc one(expand)) ; repeat*(dc one(factorize))
  end
  [] simplify => repeat*(dc one(expand)) ; repeat*(dc one(factorize))
  end
end
end
```

And now we can use it:

```
% elan poly3 someVariables.spc
enter query term finished by the key word 'end':
  deriv(3*X*X + 2*X + 7 , X) end

[] start with term: deriv(3*X*X+2*X+1,X)
[] result term:      ((X*6)+2)
[] end
```


Chapter 2

How to use ELAN

2.1 Using ELAN

The language can be used either through an interpreter or a compiler as described below in section 2.1.1 and 2.1.3 respectively.

2.1.1 How to run the ELAN interpreter

You can now execute ELAN by just typing the `elan` command with the following usage:

```
***** ELAN version 3.6
(c) LORIA (CNRS, INPL, INRIA, UHP, U-Nancy 2)
1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001
```

Usage:

```
elan [options] lgi_file [spc_file]
```

Options:

```
--dump, -d           : dump of signature, strategies and rules
--trace, -t [num]    : tracing of execution (default max)
--statistic, -s/-S   : statistics: short/long
--warningsoff, -w    : suppress warning messages of the parser
--quiet, -q          : quiet regime of execution
--batch, -b          : batch regime (no messages at all)
--elanlib            : elanlib
--secondlib, -l lib  : second elan library (by default ..)
--command, -C        : command language
--export, --cexport  : export to a .ref file
--import             : import from a .ref file
```

Both the logic description file (`lgi_file`) and the specification file (`spc_file`) may be written without their suffixes. Default suffixes for ELAN files are:

`.lgi` – for the top level logic description file,

`.eln` – for modules used in the top level logic description file,

`.spc` – for the specification file,

`.ref` – for the file in REF format.

The system ELAN searches these files (i.e. `.eln`, `.lgi`, `.spc`, `.ref`) in the following directories with the decreasing priorities:

- in the current directory,
- in the directory described by the value of a system variable `SECONDELANLIB`,
- in the directories described as `ELANLIB/commons`, `ELANLIB/noquote`, `ELANLIB/strategy` and `ELANLIB/ref`, in this order.

The default value of the variable `ELANLIB` is set to the subdirectory `elan_library/elanlib` of the source directory but it can be locally replaced by the switch `--elanlib`. The default value of the variable `SECONDELANLIB` is set to the parent directory, however, it can be also locally replaced by the switch `--secondlib` (or, `-1`) (for details, see below).

In Chapter 1, we have shown, how to simply run the ELAN interpreter with a logic description. We now illustrate on several examples the usage of the different switches.

`-d`, (or, `--dump`) dumps out the signature of the described logic (i.e. all sorts and function symbols with their profiles), definitions of all strategies and rewrite rules. For the example `poly3` from Chapter 1:

```
> elan -d poly3 someVariables
```

we obtain a list of rewrite rules of the form:

```
local rule expand:poly/poly3[Vars] [1/3/fsym=233/vars=9]
  ( VAR(0)+(( VAR(1)+ VAR(2))* ( VAR(3)+ VAR(4)))) =>
    ( VAR(0)+ ( VAR(5)+ ( VAR(6)+ ( VAR(7)+ VAR(8))))))
  where VAR(8) := (simplify:poly/poly3[Vars]) ( VAR(2)* VAR(4))
  where VAR(7) := (simplify:poly/poly3[Vars]) ( VAR(2)* VAR(3))
  where VAR(6) := (simplify:poly/poly3[Vars]) ( VAR(1)* VAR(4))
  where VAR(5) := (simplify:poly/poly3[Vars]) ( VAR(1)* VAR(3))

local rule expand:poly/poly3[Vars] [2/4/fsym=233/vars=6]
  ( VAR(0)+(( VAR(1)+ VAR(2))* VAR(3))) => ( VAR(0)+ ( VAR(4)+ VAR(5)))
  where VAR(5) := (simplify:poly/poly3[Vars]) ( VAR(2)* VAR(3))
  where VAR(4) := (simplify:poly/poly3[Vars]) ( VAR(1)* VAR(3))
. . . . . etc
```

Names of rewrite rules (e.g. `expand`) are displayed together with their sorts (e.g. `poly`), over which these rules work, and with a module name, where they have been defined (e.g. `poly3[Vars]`). An internal code of the top-most function symbol of its left-hand side and the number of variables of each rule are also displayed as an additional information. Variables appearing in rewrite rules are displayed in a uniform way: `VAR(i)`, where *i* is an internal index.

The print-out continues with a list of strategies in the form:

```
strategy global simplify:poly/poly3[Vars]
  repeat* (
    dc one(expand:poly )
  );
  repeat* (
    dc one(factorize:poly )
  )
. . . . . etc
```

The label of the strategy is composed of the name of the strategy, its sort and the name of the module where it has been defined (as for labelled rules).

The specification of the signature of the described logic is composed of a set of defined (and internal) sorts:

```

Sorts
intern int, ident, identifier, string, list[identifier],
intern ident, int, variable, intern string, bool,
poly,

```

and of a list of function symbols with their profiles in the following form:

```

Function symbols
@                : (variable)poly          pri 0 code 231;
@                : (int)poly              pri 0 code 232;
@ '+' @          : (poly poly)poly         assocRight      pri 1 code 233;
'(' @ '+' @ ')'  : (poly poly)poly         pri 0 code 233;
@ '*' @          : (poly poly)poly         assocRight      pri 2 code 234;
'(' @ '*' @ ')'  : (poly poly)poly         pri 0 code 234;
'deriv' '(' @ ',' @ ')' : (poly variable)poly pri 0 code 235;
. . . . . etc

```

-t (or, **--trace**) allows tracing ELAN programs. The maximal depth of tracing can be specified, like **-t 9**, however, by default (i.e. **-t**), it traces all levels. The number specified as the trace level counts matchings of a left-hand side, verifications of the conditional part of a rule, and evaluations of strategies applied over terms in local affectations. Thus, if the user wants to trace his/her rewrite derivation up to the 5-th level, the trace argument should be specified as **-t 15**.

```

> elan -t poly3 someVariables
. . . . . etc enter
query term finished by the key word 'end':
deriv(X*X,X) end

```

```

[] start with term :
  deriv((X*X),X)

[reduce] start:
[0] deriv((X*X),X)
    [reduce] start:
    [0] (X*deriv(X,X))
    [1] (X*1)
    [2] X
    [reduce] stop :
applying strategy 'simplify:poly/poly3[Vars]' on X
  trying 'expand:poly' on      X
  fail of 'expand:poly'
  trying 'expand:poly' on      X
  fail of 'expand:poly'
  trying 'factorize:poly' on    X
  fail of 'factorize:poly'
  trying 'factorize:poly' on    X
  fail of 'factorize:poly'
  trying 'factorize:poly' on    X
  fail of 'factorize:poly'
setting VAR(4) on X
  [reduce] start:
  [0] (deriv(X,X)*X)
  [1] (1*X)
  [2] X
  [reduce] stop :
applying strategy 'simplify:poly/poly3[Vars]' on X
  trying 'expand:poly' on      X
  fail of 'expand:poly'
  trying 'expand:poly' on      X

```

```

        fail of 'expand:poly'
        trying 'factorize:poly' on      X
        fail of 'factorize:poly'
        trying 'factorize:poly' on      X
        fail of 'factorize:poly'
        trying 'factorize:poly' on      X
        fail of 'factorize:poly'
    setting VAR(5) on X
        [reduce] start:
        [0] (X+X)
        [reduce] stop :
    applying strategy 'simplify:poly/poly3[Vars]' on (X+X)
        trying 'expand:poly' on      (X+X)
        fail of 'expand:poly'
        trying 'expand:poly' on      (X+X)
        fail of 'expand:poly'
        trying 'factorize:poly' on      (X+X)
        fail of 'factorize:poly'
        trying 'factorize:poly' on      (X+X)
        fail of 'factorize:poly'
        trying 'factorize:poly' on      (X+X)
        fail of 'factorize:poly'
    setting VAR(3) on (X+X)
    [1] (X+X)
    [reduce] stop :
    trying 'expand:poly' on      (X+X)
    fail of 'expand:poly'
    trying 'expand:poly' on      (X+X)
    fail of 'expand:poly'
    trying 'factorize:poly' on      (X+X)
    fail of 'factorize:poly'
    trying 'factorize:poly' on      (X+X)
    fail of 'factorize:poly'
    trying 'factorize:poly' on      (X+X)
    fail of 'factorize:poly'

[] result term:
    (X+X)

[] end

```

-s (**-S**, or **--statistics**) displays a brief (**-s**) or a complete (**-S**) version of statistics. These statistics contain the information about the running time of the last query, average speed in rewrites per second, the statistics of labelled and unlabelled rules, which were tried and applied.

```

> elan -S poly3 someVariables
. . . . . etc
enter query term finished by the key word 'end':
deriv(X*X,X) end

[] start with term :
    deriv((X*X),X)

[] result term:
    (X+X)

[] end

Statistics:

```

```

total time      (0.007+0.000)=0.007 sec (main+subprocesses)

average speed  714 inf/sec
               5 nonamed rules applied,    23 tried
               0  named rules applied,    20 tried

named rules
      applied   tried   rule for symbol
              0      8   expand:poly
              0     12   factorize:poly

nonamed rules
      applied   tried   rule for symbol
              0      4   ( poly + poly )
              2     12   ( poly * poly )
              3      7   deriv( poly , variable )

end of statistics

```

The complete statistics show a detailed list of applications and tries for labelled and unlabelled rules. The brief statistics do not precise the last paragraph about unlabelled rules.

- w (or, --warningsoff) eliminates all ambiguity warnings produced during parsing of the ELAN description of a logic.
- q (or, --quiet) suppresses all error messages and warnings during the execution of the ELAN program.
- b (or, --batch) suppresses all messages, thus this mode is useful when the ELAN system is executed from a script file in the batch mode.
- elanlib dir locally redefines the value of the system variable ELANLIB by the string dir.
- secondlib dir (or, --secondlib dir) does the same with the variable SECONDELANLIB.
- export fname, --cexport fname exports the input specification into the REF format stored in the file fname. The switch cexport is used only when the produced REF file is compiled by the ELAN compiler.
- import fname imports a specification in the REF format from the REF file fname produced, in general, by the switch --export.

```

> elan --export poly3.ref poly3 someVariables.spc

***** ELAN version 3.6 *****

(c)  LORIA (CNRS, INPL, INRIA, UHP, U-Nancy 2), 1994 - 2001
. . . . . etc

Export file poly3.ref has been created

> elan --import poly3.ref
***** ELAN version 3.6 *****

(c)  LORIA (CNRS, INPL, INRIA, UHP, U-Nancy 2), 1994 - 2001

Import form file poly3

Importing Identifiers
Importing Sorts

```

```

Importing Modules
Importing RuleNames
Importing StrategyNames
Import rule extractrule1:identifier/list[identifier]!L0 form module list[identifier]
Import rule extractrule2:identifier/list[identifier]!L0 form module list[identifier]
Import rule expand:poly/poly3[Vars]!L0 form module poly3[Vars]
Import rule expand:poly/poly3[Vars]!L0 form module poly3[Vars]
Import rule factorize:poly/poly3[Vars]!L0 form module poly3[Vars]
Import rule factorize:poly/poly3[Vars]!L0 form module poly3[Vars]
Import rule factorize:poly/poly3[Vars]!L0 form module poly3[Vars]
Import strategy listExtract:identifier/list[identifier]
Import strategy simplify:poly/poly3[Vars]
Import strategy last_simplify:poly/poly3[Vars]

```

enter query term finished by the key word 'end':

-C runs the ELAN interpreter with a simple command language interpreted on the top-most level. This mode is described in Section 2.1.2.

After loading all modules specified in the logic description file, ELAN requires to enter a query term, which has to be finished by the word 'end' or by character ^D (control-D). Its evaluation can be interrupted by typing ^C, the interpreter then proposes a simple run-time menu:

- **ExecutionAbort** - A - aborts the current execution and allows to enter another query term,
- **Continue** - C - continues the interrupted execution,
- **Dump** - D - dumps the signature, rules and strategies of the currently loaded logic,
- **Exit** - E - quits the ELAN system,
- **Statistics** - S, s - writes the actual statistics corresponding to the last evaluated query,
- **ChangeTrace** - T - changes trace level of the current derivation such that the interpreter asks for a new trace level,
- **ChangeQuiet** - Q - switches between two displaying modes: quiet – unquiet,

2.1.2 Top level interpreter

The command language of the ELAN interpreter improves the user's interface by offering to the user several commands, which allow to:

- change several parameters of the system,
- debug programs using break-points,
- keep the history of queries and their results,
- runs the current logic with different strategies, different entries, etc.

When the ELAN interpreter is started with the option -C, instead of asking:

enter query term finished by the key word 'end'

ELAN prompts the user:

enter command finished by ';'.

We show a brief description of all commands of the top level command language which will be later illustrated on several examples.

`quit` terminates the session and leaves the ELAN interpreter.

`load mod` loads (i.e. imports) an additional module, where the identifier `mod` describes its name (possibly with its arguments),

`batch mod` calls a script file with the name `mod` (an identifier), which contains a stream of commands of the command language. It tries to evaluate all commands of the script file, and it returns the control to the interactive mode. The script files can be also concatenated.

`qs` shows the queue of input queries (i.e. the history up to the last ten queries). The terms in this queue of queries `qs` can be referenced by the identifiers `Q`, `Q1`, ..., `Q9` respecting their sorts. These sorted identifiers could be used in constructions of further queries.

`rs` shows the queue of the latest results (i.e. the history up to the last ten results). Terms in this queue of results `rs` are referenced by identifiers `R`, `R1`, ..., `R9`. In the case of non-deterministic computations, there is no correspondence between query terms `Qi` and result terms `Ri`.

`startwith (str)term` redefines the 'start with' pattern originally defined in the logic description file `.lgi`. The pattern `term` may contain also symbols `query` standing as a place-holder of an input term entered by the user using the command `run`, or symbols `Q`, ..., `Qi` or `R`, ..., `Ri` standing for values of previously put queries or results.

`checkwith term` redefines the 'check with' pattern originally defined in the logic description file. The boolean pattern `term` may contain any symbol amongst `query`, `Q`, ..., `Qi` or `R`, ..., `Ri`.

`printwith term` sets-up the 'print-with' pattern. The 'print-with' pattern may contain the symbol `result`, which is a place-holder for result terms. Intuitively, the meaning of the 'print-with' term P is that any result r of the computation is substituted for the place-holder `result` in the pattern P . The substituted term $P[result/r]$ is normalized and printed out. This command can be used for the automatic transformation of results into different formats, e.g. a transformation of very large result terms into more readable or incomplete notation, or a conversion into a $\text{\LaTeX}$ notation. The default value of the 'print-with' pattern is `result`, i.e. the identity.

`run term` evaluates the term `term` as a query w.r.t. rewrite rules of the current logic and eventually returns results. The entry term `term` may contain identifiers `Q`, `Q1`, ..., `Q9`, (resp. `R`, `R1`, ..., `R9`) referring to values of preceding queries or results, e.g. items of the queue of queries `qs` (resp. results `rs`).

`sorts type type type` sets-up sorts of the symbols `query`, `result` and of the 'print-with' pattern.

`stat` prints the statistics,

`dump` prints rules, strategies and the signature,

`dump name` dumps only labelled rules with the name `name`, strategies labelled by `name` or unlabelled rules with the head symbol `name`.

`dump n` dumps only unlabelled rules of a function symbol with the internal code `n`.

`display n` if `n` is not zero, all printed terms are in the internal form. The internal form of a term attaches to a symbol name its internal code, which allows to identify overloaded function symbols. `display 0` switches all print-outs into traditional (thus, overloaded) format.

`trace n` sets-up the trace level to the value `n`.

`break name` (resp. `unbreak name`) sets (resp. removes) break-points to all labelled rules with the name `name`, to all strategies labelled by `name` and to all unlabelled rewrite rules with the head symbol `name`.

`break n` (resp. `unbreak n`) sets (resp. removes) break-points to unlabelled rules with the head symbol with the internal code `n`.

`breaks` shows a list of all breakpoints.

We illustrate the command language on a small ELAN session with the example `poly3` from the previous Chapter. When the switch `-C` is specified, the ELAN interpreter loads the module `Query` describing the syntax of commands.

```
> elan -q -C poly3 someVariables
```

```
***** ELAN version 3.6 *****
```

```
(c) LORIA (CNRS, INPL, INRIA, UHP, U-Nancy 2), 1994 - 2001
handling module identifier
  handling module ident
    handling module bool
    end of bool
  end of ident
  handling module eq[identifier]
    handling module cmp[identifier]
    end of cmp[identifier]
  end of eq[identifier]
end of identifier
handling module list[identifier]
  handling module int
    handling module builtinInt
    end of builtinInt
  end of int
end of list[identifier]
handling module poly3[Vars]
  handling module eq[variable]
    handling module cmp[variable]
    end of cmp[variable]
  end of eq[variable]
end of poly3[Vars]
handling module Query[poly,poly,poly]
end of Query[poly,poly,poly]
```

We can run simple queries using the command `run`:

```
enter command finished by ';':
run deriv(X*X,X);
[] start with term :
  deriv((X*X),X)
[] result term:
  (X+X)
[] end
```

```

enter command finished by ';':
run deriv(X*X*X+X*X+X+1,X);
[] start with term :
    deriv(((X*(X*X))+((X*X)+(X+1))),X)
[] result term:
    (1+(X*(((X*2)+2)+X)))
[] end

```

The queries and their results are stored in two queues, which can be listed by commands `qs`, `rs`:

```

enter command finished by ';':
qs;
Queries ... 2 : elements
Q1      poly      deriv((X*X),X)
Q       poly      deriv(((X*(X*X))+((X*X)+(X+1))),X)

enter command finished by ';':
rs;
Results ... 2 : elements
R1      poly      (X+X)
R       poly      (1+(X*(((X*2)+2)+X)))

```

The previous queries and their results can be referenced in the construction of the new ones:

```

enter command finished by ';':
run deriv(R,X) ;
[] start with term :
    deriv((1+(X*(((X*2)+2)+X))),X)
[] result term:
    ((X*6)+2)
[] end

enter command finished by ';':
run deriv(Q*Q,X) ;
[] start with term :
    deriv((deriv((1+(X*(((X*2)+2)+X))),X)*deriv((1+(X*(((X*2)+2)+X))),X)),X)
[] result term:
    (24+(X*72))
[] end

```

Function symbols defined by unlabelled rules can be debugged, for example, if we know their internal code. In the following example, the command `dump` displays the symbol with the internal code 216, and then, we can put a break-point on this function.

```

dump 235;
function dump
'deriv' '(' @ ',' @ ')' : (poly variable)poly pri 0 code 235;

enter command finished by ';':
break 235;
function dump
'deriv' '(' @ ',' @ ')' : (poly variable)poly pri 0 code 235;

```

Having break-points set, the execution shows all entry and exit points of traced functions, or strategies:

```

enter command finished by ';':
run deriv(X*X,X);

[] start with term :
    deriv((X*X),X)
[0] deriv((X*X),X)

```

```

[0] deriv(X,X)
[1] 1
[0] deriv(X,X)
[1] 1
[1] (X+X)
[] result term:
  (X+X)
[] end

```

We can switch all outputs into the internal format by the command `display 1`:

```

enter command finished by ';':
display 1;

enter command finished by ';':
run deriv(X*X,X);
[] start with term :
  deriv_235((X_228*X_228),X_228)
[0] deriv_235((X_228*X_228),X_228)
    [0] deriv_235(X_228,X_228)
    [1] 1
    [0] deriv_235(X_228,X_228)
    [1] 1
[1] (X_228+X_228)
[] result term:
  (X_228+X_228)
[] end

```

We can change the 'start-with' pattern to `(last_simplify)deriv(query,X)`, and the command `stat` informs us about the current setting and the performance of the last executed query.

```

enter command finished by ';':
startwith (last_simplify)deriv(query,X);

enter command finished by ';':
stat;
Input type : poly
Output type: poly
Print type : poly
Strategy   : last_simplify
Start_with : deriv_235( VAR(0),X_228)
Check_with : true_1
Print_with : VAR(0)
-----

Statistics:
total time      (0.013+0.010)=0.023 sec (main+subprocesses)

average speed  384 inf/sec
                5 nonamed rules applied,    23 tried
                0  named rules applied,    20 tried
named rules
    applied    tried    rule for symbol
        0         8    expand:poly
        0        12    factorize:poly
nonamed rules
    applied    tried    rule for symbol
        0         4    ( poly + poly )
        2        12    ( poly * poly )
        3         7    deriv( poly , variable )
end of statistics

```

2.1.3 How to run the ELAN compiler

The ELAN compiler transforms a logic description into an executable binary file. The compiler (for detailed description, see [Vit96, MK98]) produces an efficient C code, which is later compiled by the `cc` or the `gnu gcc` compiler. The executable binary code is dependent on the particular architecture, because a small part of the ELAN library performing the basic non-deterministic operations is written in assembly language [Mor98]. Up to now, this is the only reason that makes the compiler available only on the architectures DEC-ALPHA, SUN4 and Intel-PC. This release of the ELAN compiler does not compile the whole ELAN language: the main restriction is that the ELAN compiler does not treat input/output and reflective capabilities of the ELAN language. There is also a restriction on the class of rules containing AC-symbols that can be compiled. Thus, any logic description containing AC-symbols should be carefully studied before being compiled (see Section 4.5 for more details).

To run the ELAN compiler, a Java Virtual Machine (JAVA 2 SDK, aka jdk 1.2, or newer), a C compiler (`gcc` recommended) and a C-shell (`tcsh` recommended) have to be installed on your system. To complete the installation, the script `PWD/bin/elanc` has to be modified: `CLASSPATH` and `JAVA` variables have to be set according to your system. Then, you can now execute the ELAN compiler by just typing the `elanc` command with the following usage:

```
elanc [[elan options][compiler options]] file[.lgi] [file.[spc]]
```

Options:

```
-output <name>
-verbose
-nocode
-nosplit
-quiet
-debug
-noOptimiseChoicePoint [default]
-optimiseChoicePoint
-proofterm
```

By default, the compiler produces a lot of C files and a `lgiName.makefile` file, which are later compiled.

The switch `-output nqueens` produces the executable file with the name `nqueens`.

The switch `-nosplit` generates only two files: `lgiName.c` and `lgiName.core.c`. This switch is useful to compile small specifications.

The switch `-nocode` directly builds an executable file and deletes all generated C files.

The switch `-quiet` suppresses all printed messages during compilation phase.

The switch `-debug` activates the debugging mode: more messages are printed during compilation and the generated code becomes less efficient.

The switch `-noOptimiseChoicePoint [default]` avoids the deterministic analysis phase.

The switch `-optimiseChoicePoint` reduces the number of set choice-points: this switch reduces the runtime needed memory and improve the efficiency. It is recommended to not activate this switch if your specification contains AC symbols.

The switch `-proofterm` generates the complete trace of the normalisation process with respect to a set of labelled rules by the strategy **normin** described in Section 3.

We illustrate the compiler on a simple example of a distributed file. The first example shows the compilation of the program `nqueens` with the query `queens(8)` (given in the `.lgi` file):

```
> elanc -output queens -nosplit -nocode -optimiseChoicePoint nqueens
elan --cexport nqueens.ref ./nqueens.lgi

***** ELAN version 3.6 *****

(c) LORIA (CNRS, INPL, INRIA, UHP, U-Nancy 2), 1994 - 2001
handling module nqueens
  handling module int
    handling module builtinInt
      handling module bool
        end of bool
      end of builtinInt
    end of int
  end of nqueens

Export file nqueens.ref has been created

java -classpath classes.zip:elanlib/Compiler/classes REM nqueens.ref
-output queens -nosplit -nocode -optimiseChoicePoint
Reduce ELAN Code Machine. Reading from file nqueens.ref . . .
Parsing.....
Compiling nonamed rules.....
Compiling strategies..
Building queens...

> queens
result = .([ (4),.([ (2),.([ (7),.([ (3),.([ (6),.([ (8),.([ (5),.([ (1),nil)))))])
result = .([ (5),.([ (2),.([ (4),.([ (7),.([ (3),.([ (8),.([ (6),.([ (1),nil)))))])
result =
.([ (3),.([ (5),.([ (2),.([ (8),.([ (6),.([ (4),.([ (7),.([ (1),nil)))))])
...
after 91 solutions:
result = .([ (5),.([ (7),.([ (2),.([ (6),.([ (3),.([ (1),.([ (4),.([ (8),nil)))))])

rewrite_step = 227356
```

It is also possible to get the whole trace of the execution. In order to produce this information the executable code has to be compiled in the debugging mode and the switch `-trace` has to be used:

```
> elanc -debug -nosplit nqueens
...
Reduce ELAN Code Machine. Reading from file nqueens2.ref . . .
Parsing
Compiling nonamed rules.....
Compiling strategies..
execute the following line to build your program
  gmake -f nqueens.make
> gmake -f nqueens.make
gcc -pipe -g3 -DDEBUG -DPDEBUG -DBITSET32 ...
...

> a.out -trace
|start with: fun_221_queens()_([ (8))
|rewrite[1] queens(,)([ (8),[ (8),nil)
|start with: str_176(queens(,)([ (8),[ (8),nil))
| start with: fun_211_greater_int(,)_([ (8),[ (0))
```

```

| rewrite[2] true
| start with: str_278([] (8))
| rewrite(str_278)[3] [] (1)
| start with: fun_222_noattack(, ,)_([] (1), [] (1), nil)
| rewrite[4] true
| start with: fun_202_minus(, ,)_([] (8), [] (1))
| rewrite[5] [] (7)
| start with: str_176(queens(, ,)_([] (7), [] (8), .([] (1), nil)))
| start with: fun_211_greater_int(, ,)_([] (7), [] (0))
| rewrite[6] true
| start with: str_278([] (8))
| rewrite(str_278)[7] [] (1)
| start with: fun_222_noattack(, ,)_([] (1), [] (1), .([] (1), nil))
| start with: fun_210_neq_int(, ,)_([] (1), [] (1))
| rewrite[8] false
| rewrite[8] noattack(, ,)_([] (1), [] (1), .([] (1), nil))
...

```

A more detailed description of the compiler can be found in Section 4.5.

▷ Another way to call the ELAN compiler (also known as *Reduced ELAN Machine*) consists in running Java as follows:

```
java -classpath ${ELANLIB}/Compiler/classes REM <REF file name> <options>
```

The <REF file name> must be created by the option `--cexport <REF file name>` of the ELAN interpreter simply used here to parse the <LGI file name> .

2.2 Keep in touch

In case you have any problems, questions or comments about ELAN, or just only to be keeping in touch with the ELAN team, please use the following e-mail address:

elan@loria.fr or post on the elan-users mailing-list:

elan-users@loria.fr

More informations on ELAN can be found on the web page:

<http://elan.loria.fr>.

ELAN is issued from the PROTHEO research team, information about this research project can be found at the following address:

<http://www.loria.fr/equipes/protheo>.

Chapter 3

ELAN: the language

This chapter presents a full bottom-up description of the language features that, contrary to Chapter 1, uses a construction of the language only when it has been introduced before.

The ELAN syntax is given in the Extended Backus-Naur Form (EBNF). We assume that the reader is familiar with this well-known notation, where for instance $\{ X \}^+$ (resp. $\{ X \}^*$) denotes a non-empty (resp. a possibly empty) repetition of X, and $[X]$ means that X is optional, and may be omitted.

All examples given below are running under the current implementation of the ELAN version V3.6.

3.1 Lexicographical conventions

3.1.1 Separators

All ASCII characters whose code are less or equal than 32 are considered by the parser as *separators*. Thus spaces, tabulations and newlines are separators.

3.1.2 Lexical unities

A *lexical unity* in ELAN is either an identifier, a natural number or a special character.

Identifiers are composed by concatenation of letters, numerals and the character “_”, and they should begin with a letter.

$\langle letter \rangle ::= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z$
 $\quad \quad \quad | \quad a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z$

$\langle numeral \rangle ::= 0|1|2|3|4|5|6|7|8|9$

$\langle identifier \rangle ::= \langle letter \rangle \{ \langle letter \rangle | \langle numeral \rangle | _ \}^*$

$\langle quoted\ identifier \rangle ::= ' \{ \langle letter \rangle | \langle qchar \rangle | \langle numeral \rangle | _ \}^+ '$

$\langle elan\ identifier \rangle ::= \langle identifier \rangle$ except ELAN's keywords

All characters different from the previously introduced ones (letters, numerals, separators) and the characters ‘{’, ‘}’, ‘~’, are considered as special characters and are referenced as $\langle char \rangle$.

$\langle nchar \rangle ::= \langle char \rangle$ except ‘@’ and ‘.’

$\langle qchar \rangle ::= \langle char \rangle \text{ except '}'$

A *number* is the concatenation of numerals.

$\langle number \rangle ::= \{ \langle numeral \rangle \}^+$

3.1.3 Comments

All strings enclosed between the strings “/*” and “*/” or between the string “//” and the end of line, are considered by the parser as *comments*.

3.2 Element modules

Element modules are the basic unit blocks of the language. They allow to define computational systems i.e. to define a rewrite theory together with a strategy. For modularity reasons, importations are made possible in ELAN modules, under the condition that no cycle exists in the importation relation. This is described in the modularity section 3.7.

The syntax is the following:

```

<module> ::= module <formal module name>
           [ <imports> ]
           [ <sort definition> ]
           [ <operator definition> ]
           [ <stratop definition> ]
           { <family of rules> } *
           { <family of strategies> } *
           end

```

The module of name `ModuleName` should be declared in the file `ModuleName.eln`. Only one, and exactly one module can be described in a given file.

Importations can be made local or global:

```

<imports> ::= import { <instantiated module name> } + ; end
           | import [ <global imports> ] [ <local imports> ] end

```

```

<global imports> ::= global { <instantiated module name> } + ;

```

```

<local imports> ::= local { <instantiated module name> } + ;

```

An instantiated module name like `List[List[Int]]` is described by:

```

<instantiated module name> ::= <identifier>
                             | <identifier> [ <instantiated module name>
                                                { , <instantiated module name> } * ]

```

Sorts are always global:

```

<sort definition> ::= sort { <sort name> } + ; end

```

but operators can be exported, and thus declared global, or just local in which case they can be used only in the module where they are defined.

```
<operator definition> ::= operators
                        [ <global operator definition> ] [ <local operator definition> ]
                        end
```

```
<global operator definition> ::= global { <operator declaration> } + ;
```

```
<local operator definition> ::= local { <operator declaration> } + ;
```

And similarly for strategy operator definition:

```
<stratop definition> ::= stratop
                        [ <global stratop definition> ] [ <local stratop definition> ]
                        end
```

```
<global stratop definition> ::= global { <strategy declaration> } + ;
```

```
<local stratop definition> ::= local { <strategy declaration> } + ;
```

<family of rules> is defined in Section 3.4. <family of strategies> is defined in Section 3.5.

3.3 Definition of signatures

ELAN is a multi-sorted language, and signatures are composed by the definitions of the sorts and operators symbols. The operator syntax is given in a mix-fix form which allows the super-user to define very conveniently its own syntax.

3.3.1 Sort declarations

A *sort declaration* is either a sort name (an identifier) or a sort name followed by the list of sort names it depends on:

```
<sort name> ::= <identifier>
              | <identifier> [ <sort name> { , <sort name> } * ]
```

Example 3.1 `bool, int, list[bool], pair[int,list[bool]], list[X]` are sort names in ELAN.

Remark: A sort name is always *global* in ELAN. Two sort declarations using the same sort name, will result in the creation of only *one* sort.

Built-in sorts For efficiency, sorts could be defined as built-ins. See section 5.2 for details.

3.3.2 Operator declarations

A *function symbol*, also called *operator*, is given either by defining a new symbol together with its rank or by aliasing an existing operator. Optional properties can be specified:

$$\begin{aligned} \langle \text{operator declaration} \rangle & ::= \langle \text{new operator declaration} \rangle \\ & \quad | \quad \langle \text{operator alias} \rangle \end{aligned}$$

$$\langle \text{new operator declaration} \rangle ::= \langle \text{operator name} \rangle : \langle \text{rank} \rangle [\{ \langle \text{operator option} \rangle \}^+]$$

$$\langle \text{operator alias} \rangle ::= \langle \text{new operator declaration} \rangle \text{ alias } \langle \text{old operator name} \rangle :$$

In case of *aliasing*, $\langle \text{new operator declaration} \rangle$ is only another name for the function symbol $\langle \text{old name} \rangle$. The *two* names are then accessible and refer to the *same* object.

▷ The last alias defined is considered by the parser as having the highest priority with respect to the previously defined ones.

$$\langle \text{old operator name} \rangle ::= \langle \text{operator name} \rangle$$

$$\langle \text{operator name} \rangle ::= \langle \text{name} \rangle$$

The name of a function symbol contains information that will be used by the mixfix parser to read a term:

$$\langle \text{name} \rangle ::= \{ \langle \text{symbol} \rangle \}^+$$

$$\begin{aligned} \langle \text{symbol} \rangle & ::= \langle \text{single lexem} \rangle \\ & \quad | \quad @ \\ & \quad | \quad \langle \text{quoted single lexem} \rangle \\ & \quad | \quad '\backslash \langle \text{number} \rangle ' \end{aligned}$$

$$\begin{aligned} \langle \text{single lexem} \rangle & ::= \langle \text{elan identifier} \rangle \\ & \quad | \quad \langle \text{number} \rangle \\ & \quad | \quad \langle \text{nchar} \rangle \end{aligned}$$

$$\langle \text{quoted single lexem} \rangle ::= \langle \text{quoted identifier} \rangle$$

where, as defined on page 29, $\langle \text{elan identifier} \rangle$ is any identifier except keywords. Notice that indeed keywords can be used without their special meaning when placed between quotes.

The characters ‘{’, ‘}’, ‘~’ are restricted to the pre-processor use. Thus they cannot be used in any module, but they can be used in the specification file or in the description of the query, since both are not pre-processed. The ‘.’ character indicates the end of the name and the character ‘@’ is the place-holder for an operator ‘@’argument. These two last characters can be used with another semantics than their built-in one, when placed between quotes.

A quoted number after the character ‘\’ represents the definition of a character by its ascii code. This facility can be used in order to add to the syntactic form of a term non-visible characters or the characters ‘{’, ‘}’, ‘~’.

Finally an operator declaration is achieved by defining its *rank*, which is just a list of sort names with optional description of *selectors*.

$$\begin{aligned}
\langle \text{rank} \rangle &::= \langle \text{sort name} \rangle \\
&| \left(\{ \langle \text{sort name} \rangle \}^+ \right) \langle \text{sort name} \rangle \\
&| \left(\{ \langle \text{identifier} \rangle : \langle \text{sort name} \rangle \}^+ \right) \langle \text{sort name} \rangle
\end{aligned}$$

Example 3.2 *With the two declarations:*

<code>@ + @</code>	<code>: (int int) int</code>	
<code>+(@,@)</code>	<code>: (int int) int</code>	<code>alias @+@:</code>

the strings “x + y” and “+(x,y)” represent the same term.

When defining a constructor operator, it is possible to define the related selectors and modifiers, as outlined in the next example.

Example 3.3 *In the following module example, the selector `first` and `second` are specified and the system will automatically generate the corresponding selector and modifier operators and the corresponding rewrite rules.*

```

module simplePairSelectors
import bool;
end
sort X Y Pair;
end
operators
global
  [@,@]      : ( first:X second:Y ) Pair;
  isPair(@)  : ( Pair ) bool;
end
end

```

This is similar to the following module:

```

module simplePair
import bool;
end
sort X Y Pair;
end
operators
global
  [@,@]      : ( X Y ) Pair;
  isPair(@)  : ( Pair ) bool;
  \\\ automatically generated selectors
  @.first    : (Pair) X;
  @.second   : (Pair) Y;
  \\\ automatically generated modifiers
  @[.first<-@] : (Pair X) Pair;
  @[.second<-@] : (Pair Y) Pair;
end
end

```

where the following rewrite rules are in fact automatically generated:

```

(x,y).first => x
(x,y).second => y
(x,y)[.first<-z] => (z,y)
(x,y)[.second<-w] => (x,w)

```

3.3.3 Function declaration options

As in languages like OBJ or ASF+SDF, an operator declaration carries many informations about the syntax and the semantics of the operator. We also need to specify complementary information like the precedence of the operator for the parser, or some of its semantic properties like associativity. This is done in ELAN using the following syntax:

```
<operator option> ::= assocLeft
                    | assocRight
                    | pri <number>
                    | ( AC )
                    | code <number>
                    | builtin <number>
```

Parsing Associative operators The first two options are useful for defining syntactic associativity of symbols:

Example 3.4 *The declarations:*

```
@ * @          : ( int int ) int      assocLeft
*(@,@)         : ( int int ) int      alias @*@:
```

*allow to parse the term $x * y * z$ as $*(x, y, z)$.*

Priorities The *priority* parsing problems can be solved using the priority option:

Example 3.5 *The declarations:*

```
@ + @          : ( int int ) int      assocLeft pri 10
@ * @          : ( int int ) int      assocLeft pri 20
```

*allow to parse an expression like $x + y * z$ as usual in $x + (y * z)$.*

Coercions can be defined and even hidden as in the following example:

Example 3.6 *Assume that equations are constructed with the equality symbol:*

```
@ = @          : ( term term ) equation;
```

and equational systems as conjunctions of equational systems:

```
@ & @          : ( eqSystem eqSystem ) eqSystem;
```

*An equation should be considered as an equational system, which corresponds to consider the sort **equation** as a subsort of **eqSystem**. This is done by the declaration of an invisible operator:*

```
@              : ( equation ) eqSystem;
```

This (invisible) coercion allows parsing the expression $(P \& e)$ where $P:\text{eqSystem}$; $e:\text{equation}$.

Semantic properties Semantic properties can be used. Currently only associativity and commutativity (abbreviated *AC*) is allowed and only for binary operators. The user should be aware that in interpreted mode, *AC* matching can be quite inefficient. It is strongly recommended to use the compiled mode in presence of *AC* operators.

Example 3.7 *One can define a union operator which is associative and commutative in the following way:*

```
@ ∪ @          : ( set set ) set      (AC)
```

User built-in operators In order to improve efficiency of the evaluation, built-in operators can be specified using the **code** and **builtin** options. The natural number specified after the **code** and **builtin** keywords specifies the index of this symbol in the symbol table of ELAN. A **builtin** option is for built-in operators without attached code defining its semantics, while the **code** option is for built-in operators with an attached procedure.

Example 3.8 The symbol **true** is defined in the **bool** module as the first symbol of the symbol table:

```

true                : bool                builtin 1

```

Section 5.2 summarizes the codes already defined for ELAN's standard built-ins.

▷ Adding built-in operators is of course a delicate matter since the ELAN source code has to be modified.

3.3.4 Strategy declaration

A *strategy operator* is similar to a function symbol as defined previously except that at least one of the sorts involved in the strategy rank is a strategy sort.

```

<strategy declaration> ::= <new strategy declaration>
                        | <strategy alias>

```

```

<new strategy declaration> ::= <strategy name> : <strategy rank> [ <strategy option> ]

```

```

<strategy alias> ::= <new strategy declaration> alias <old strategy name> :

```

In case of *aliasing*, *<new strategy declaration>* is only another name for the strategy operator *<old strategy name>*. The two names are then accessible and refer to the *same* object.

▷ The last alias defined is considered by the parser as having the highest priority with respect to the previously defined ones.

```

<old strategy name> ::= <strategy name>

```

```

<strategy name> ::= <name>

```

```

<strategy rank> ::= <strategy sort name>
                  | ( <strategy args> ) <sort name>
                  | ( { <strategy arg> } + ) <strategy sort name>

```

```

<strategy arg> ::= <sort name>
                | <strategy sort name>

```

```

<strategy args> ::= <sort name> <strategy args>
                  | <strategy sort name> { <strategy arg> } *

```

```

<strategy sort name> ::= < <strategy sort name> -> <strategy sort name> >
| < <sort name> -> <sort name> >
| < <sort name> -> <strategy sort name> >
| < <strategy sort name> -> <sort name> >

<strategy option> ::= <elementary strategy option>
| <defined strategy option>

<elementary strategy option> ::= bs

<defined strategy option> ::= assocLeft
| assocRight
| pri <number>
| ( AC )

```

The **bs** attribute (for **b**asic **s**trategy) indicates that the definition of the strategy is given by a rewrite rule whose right-hand side only uses the constructions of the elementary strategies language defined in Section 3.5.1.

3.4 Definition of rules

Rewriting is the basic concept underlying the design of ELAN. The language allows defining labelled conditional rules with local assignments. It is important to realize the difference between:

- **labelled rules** whose evaluation is fully controled by the user defined strategies and,
- **unlabelled rules** which are intended to perform functional evaluation. They are applied using a built-in left-most inner-most strategy.

A *condition* controlling the firing of a rule is a term of sort **bool** introduced by the **if** keyword. A *local assignment* is introduced by the keyword **where**. Its main purpose is to assign to local variables the result of the application of a strategy on a term.

3.4.1 Rule syntax

Rules are introduced in families, possibly restricted to a single element.

```

<family of rules> ::= rules for <sort name>
| [ { <variable declare> } + ]
| [ global { <labelled or unlabelled rule> } + ]
| [ local { <labelled rule> } + ]
| end

<labelled or unlabelled rule> ::= <labelled rule>
| <unlabelled rule>

<labelled rule> ::= [ <rule label> ] <rule body> end

<rule label> ::= <identifier> [ ( variable { , <variable> } * ) ]

<unlabelled rule> ::= [ ] <rule body> end

```

```

<rule body> ::= <term> ==> <term> [ { <if-where-choose> } * ]

<if-where-choose> ::= if <boolean term>
  | where <variable name> := ( [ <Estrategy term> ] ) <term>
  | where <variable name> := [ <Dstrategy term> ] <term>
  | where ( <sort name> ) <term> := ( [ <Estrategy term> ] ) <term>
  | where ( <sort name> ) <term> := [ <Dstrategy term> ] <term>
  | choose
    { try <if-where-choose> } +
    end

<variable declare> ::= [ { <variable name> , } * <variable name> ] : <sort name> ;

<variable name> ::= <elan identifier>

```

<Estrategy term> defined in Section 3.5.1 is a constant strategy name declared with the attribute **bs**. Note that it is optional.

<Dstrategy term> defined in Section 3.5.2 is a term built on the signature of strategy operators.

Remark: The syntax for terms is not defined here since it is user defined and induced from the rank of the operators. Because of the very liberal way of defining operator ranks, the term syntax in *mixfix* in the tradition of algebraic specification languages like ASF+SDF or OBJ. The term parser is based on Earley’s context free parser [Ear70] where the grammar is extracted from the operator rank definitions.

— **Release Note:** Since *V.3.0*, the *choose-try* construction, the *patterns in where* facility and the user defined strategies call by “[]” have been introduced.

The rule labels accept now arguments that should be variables used in the rule.

In the **where** construction with a term, the sort is just used to declare the sort of the term used as pattern. This is useful for the parser which then just checks that this is also the sort of the result on the right hand side of “:=”. Note that the sort declarations is useless for the **where** construction using only variables, since the sort of the variable is always attached to it.

There are two ways for calling strategies, according to the strategy language used by the programmer:

▷ A strategy call of the form “()” can only be done using a strategy that is declared as **bs**.

▷ A strategy call of the form “[]” can only be done using a strategy that is *not* declared as **bs**.

Example 3.9 Here is one way for defining the idempotency rules for booleans. Notice that the two rules are local to the module so that their label can only be used for defining strategies in this module. Note also that the two rules have, on purpose, the same label:

```

rules for boolean
  x : boolean;
local
[idempotency]  x and x => x  end
end

rules for boolean
  x : boolean;
local
[idempotency]  x or x  => x  end
end

```

Since several rules may have the same label, the resulting ambiguities are handled by the strategy evaluation as explained later. The rule label is optional and rules may have no name. This last case is designed in such a way that the intended semantics is functional, and it is currently the responsibility of the rewrite rule designer to check that the rewrite system composed of all unlabelled rules is confluent and terminating. Unlabelled rules are handled by a built-in strategy (left-most inner-most) as fully described in Section 3.6.

3.4.2 The where construction

The **where** construction has several purposes. The first one is to modularize the computations by computing auxiliary results affecting them to local variables, thus providing computation sharing. The second is to direct the evaluation mechanism using strategies and the built-in left-most inner-most evaluation reduction. Let us review the possibilities offered by ELAN.

The **where** statement is first useful when one wants to call a strategy on subterms.

Example 3.10 *As a first introducing example, let us consider the module:*

```
module ruleWithWhere
  sort foo; end
  operators global
    a : foo;
    b : foo;
  end
  stratop global
    strat : <foo -> foo>      bs;
  end
  rules for foo
    x : foo ;
    global
      [r0]      a => b                      end
      [r1]      a => x where x := ()b        end
      [r2]      a => x where x := (strat)b    end
    end
  // ....
end
```

Using the rule **r0**, the term **a** reduces in one step to **b**. Using the rule **r1**, the term **b** is first normalized into **b'** using unlabelled rules and then **a** is replaced by **b'**. Using the rule **r2**, the term **b** is first normalized into **b'** using unlabelled rules and then the strategy **strat**, defined elsewhere by the super-user, is applied on **b'** and the result is substituted to **a**.

One can also perform more complex affectations of the local variables by using *patterns*.

The current syntax of the **where** statement with patterns can be rephrased as follows:

$$\mathbf{where} \ (sort)p ::= ()term \mid (s)term \mid [s]term$$

where the pattern *p* should be of sort *sort*. AC-operators as well as non-linearity are allowed for patterns.

This pattern in where capability is exemplified in the next subsection.

3.4.3 Choose-Try

The construction **Choose try** allows factorizing common parts of rules with the same left-hand side. For instance the two rules

$$\begin{array}{l}
 [\ell] \quad l \Rightarrow r \quad \textbf{where } y_1 := ()u_1 \\
 \quad \quad \quad \textbf{if } c'_2(l, y_1) \\
 [\ell] \quad l \Rightarrow r \quad \textbf{where } y_1 := ()u_1 \\
 \quad \quad \quad \textbf{if } c''_2(l, y_1) \\
 \quad \quad \quad \textbf{where } y_3 := ()u_3
 \end{array}$$

are factorized into one rule:

$$\begin{array}{l}
 [\ell] \quad l \Rightarrow r \quad \textbf{where } y_1 := ()u_1 \\
 \quad \quad \quad \textbf{choose} \\
 \quad \quad \quad \quad \textbf{try if } c'_2(l, y_1) \\
 \quad \quad \quad \quad \textbf{try if } c''_2(l, y_1) \\
 \quad \quad \quad \textbf{end} \\
 \quad \quad \quad \textbf{where } y_3 := ()u_3
 \end{array}$$

In this factorized form, the term u_1 is normalized only once and the assignment to y_1 is performed also only once.

Example 3.11 *This example shows the use of patterns as well as the **Choose try** construction.*

```

module quick
  import global int list[int]; end
  sort pair; end
  operators global
    sort(@)          : (list[int]) list[int];
    pivot(@,@,@,@)   : (int list[int] list[int] list[int]) pair;
    [@,@]            : (list[int] list[int]) pair;
  end
  rules for list[int]
    x      : int;
    xs,s,l : list[int];
  global
    [] sort(nil) => nil end
    [] sort(x.xs) => append(sort(s),x.sort(l))
                      where (pair)[s,l]:=()pivot(x,xs,nil,nil) end
  end
  rules for pair
    p, x      : int;
    xs,s,l, l1, l2 : list[int];
  global
    [] pivot(p,nil,s,l) => [s,l] end
    [] pivot(p,x.xs,s,l) => pivot(p,xs,l1,l2)
                          choose try where l1 := ()s
                                where l2 := ()x.l
                                if p < x
                                try where l1 := ()x.s
                                where l2 := ()l
                                if not(p < x)
                                end
  end
end
end
end

```

3.4.4 Conditions

The boolean term c given in a condition is reduced using the strategy that results from normalization using the non labelled rules and by the user defined strategies. If the computation stops and the resulting term is the boolean **true** then the condition is considered to be satisfied and the rule can be applied. If the computation stops and the resulting term is not **true** then the condition is considered as false.

3.4.5 Labels visibility

The **global** and **local** attributes make sense only for labelled rules. When a labelled rule is:

- **local**, then its label can only be used in the module in which the rule is declared,
- **global**, then its label can be used in all the modules importing the module defining the rule, with the visibility described in the section on modularity (see section 3.7).

▷ According to their semantics, unlabelled rules are always global.

— Release Note: *Since version V.3.0, the notion of single rule has been removed and the notion of rule locality has been introduced, so that rule labels can now be declared to be local to a module or, on the contrary, globally accessible. See section 3.9 for details concerning these changes.*

Remark: We will see next on page 60 how rule families can be constructed automatically in ELAN using the pre-processor construction “FOR EACH”.

3.5 Definition of strategies

The notion of strategy is one of the main originalities of ELAN. In practice, a strategy is a way to describe which rewrite computations the user is explicitly interested in. It specifies when a given rule should be applied in the term to be reduced. From a theoretical point of view, a strategy can be viewed either as a function or as a subset of all proof terms defined by the current rewrite theory. The application of a strategy to a term results in the (possibly empty) collection of all terms that can be derived from the starting term using this strategy [KKV95, Vit94]. When a strategy returns an empty set of terms, we say that it *fails*.

▷ A strategy enumerates all the terms it describes, should this collection be finite or not. Consequently the user should note that in case (s)he writes a strategy that enumerates an infinity of terms, then the evaluation process will of course not terminate.

Two strategy languages are offered in ELAN. The language of *elementary strategies* is mainly based on regular expressions built from rule labels, while the language of *defined strategies* allows the user to define his own elaborated strategies, possibly recursively. Although the language of defined strategies embeds all the constructions of the language of elementary strategies, the latter is maintained since it is based on a more efficient implementation.

Families of strategy rules are intended to give a semantics to the strategy operators. The syntax of strategy rule families is the following:

```
<family of strategies> ::= strategies for <sort name>
                        [ { <strategy variable declare> } + ]
                        { implicit { <elementary strategy rule> } +
                        | implicit { <defined strategy rule> } +
                        | explicit { <defined strategy rule> } + } +
                        end
```

```

<strategy variable declare> ::= [ { <variable name> , } * <variable name> ]
                               :
                               <sort or strategy sort name> ;

<sort or strategy sort name> ::= <sort name>
                               | <strategy sort name>

```

The keyword **explicit** is only used with defined strategies and introduces a strategy rule that contains explicitly in its left-hand side the application operator $[@]@$ that will be introduced in Section 3.5.2. Otherwise, the strategy rule is introduced by the keyword **implicit**.

3.5.1 Elementary strategies syntax

The general syntax of elementary strategy rules is the following:

```

<elementary strategy rule> ::= [] <Estrategy name> ==> <Estrategy term> end

```

An elementary strategy rule has no label. Thus it is always global.

```

<Estrategy term> ::= <Estrategy name>
                  | <choosing>
                  | <concatenation>
                  | <iterator>
                  | <normalize>
                  | <normalize with trace>
                  | fail
                  | id

<choosing> ::= dc ( <Estrategy term> { , <Estrategy term> } * )
              | dk ( <Estrategy term> { , <Estrategy term> } * )
              | first ( <Estrategy term> { , <Estrategy term> } * )
              | dc one ( <Estrategy term> { , <Estrategy term> } * )
              | first one ( <Estrategy term> { , <Estrategy term> } * )

<concatenation> ::= <Estrategy term> ; <Estrategy term>

<iterator> ::= iterate* ( <Estrategy term> )
              | iterate+ ( <Estrategy term> )
              | repeat* ( <Estrategy term> )
              | repeat+ ( <Estrategy term> )

<normalize> ::= normalize ( <Estrategy term> )
              | normalise ( <Estrategy term> )

<normalize with trace> ::= normin ( <Estrategy name> { , <Estrategy name> } * )

```

$\langle Estrategy name \rangle$ is a constant strategy name declared with the attribute **bs**.

Labelled rules

A labelled rule is the most elementary strategy and is called a *primal strategy*.

The application of a rewrite rule in ELAN yields a set of results. There are in general several ways to apply a given conditional rule with local assignments. This is first due to equational matching (e.g. AC-matching) and second to the **where** assignment, since it may itself recursively return several possible assignments for variables when using for example non-deterministic strategies.

Note that there may be several rules with the same label. If no rule labelled ℓ applies on the term t , the set of results is empty and we say that the rule ℓ fails.

A labelled rule ℓ can be considered as the simplest form of a strategy which returns all results of the rule application.

Thus the language provides basic constructions to handle this non-determinism through choice strategy operators.

Choice strategy operators

Let us first concentrate on expressing choices among the set of results on a primal strategy ℓ . By default the strategy ℓ returns all results of this primal strategy. If at most one result is needed, ℓ has to be encapsulated by the strategy operator **dc one** that returns a non-deterministically chosen result, or **first one** that returns the first result, when it exists.

Example 3.12 *Let us consider the following module (incompletely described here):*

```
empty      :          set;
@          : ( elem )  set;
@ U @      : ( set set ) set      (AC);
element(@): ( set )   elem;

rules for elem
  S : set;
  e : elem;

local
  [extractrule] element(S U e) => e      end
end
```

The application of the rule **extractrule** on the term **empty U a U b** can be done in two ways, since the AC-matching algorithm returns a complete set of matches consisting in the two substitutions: $\{ S \mapsto \text{empty U a}, e \mapsto \text{b} \}$ and $\{ S \mapsto \text{empty U b}, e \mapsto \text{a} \}$.

Using the strategy **extractrule** on this term results in the set consisting of two terms **a** and **b**. **dc one(extractrule)** returns either **a** or **b** depending of the implementation of the non deterministic choice. **first one(extractrule)** returns either **a** or **b** depending of the implementation of the AC-matcher.

More choice operators are defined and apply in general on a list of arguments.

- The **dk** operator, with a variable arity, is an abbreviation of *dont know choose*. **dk**($S_1, \dots, S_n$) takes all strategies given as arguments, and returns, for each of them the set of all its results. **dk**($S_1, \dots, S_n$) fails if all strategies $S_1, \dots, S_n$ fail.
- The **dc** operator, with a variable arity, is an abbreviation of *dont care choose*. **dc**($S_1, \dots, S_n$) selects only one strategy that does not fail among its arguments, say S_i , and returns all its results. **dc**($S_1, \dots, S_n$) fails if all strategies $S_1, \dots, S_n$ fail. How to choose S_i is not specified.

- A specific way to choose an S_i is provided by the **first** operator that selects the first strategy that does not fail among its arguments, and returns all its results. So if S_i is selected, this means that all strategies $S_1, \dots, S_{i-1}$ have failed. Again **first**($S_1, \dots, S_n$) fails if all strategies $S_1, \dots, S_n$ fail.
- If only one result is wanted, one can use the operators **first one** or **dc one** that select a non-failing strategy among their arguments (either the first or anyone respectively), and return a non-deterministically chosen result of the selected strategy.

Example 3.13 *Consider the following module:*

```

module testStrat
sort foo;
end
operators
global
  a : foo;   b : foo;   c : foo;
  d : foo;   e : foo;   f : foo;
end
stratop
global
  strat : <foo -> foo>      bs ;
end
rules for foo
global
[r1] a      => c      end
[r2] a      => d      end
[r3] b      => e      end
[r4] b      => f      end
[r5] c      => b      end
end

strategies for foo
implicit
[] strat => dc(dk(r1), dk(r2), dk(r3, r4)) end
end
end

```

Then the strategy **strat** applied to **a** results in **c**, and the same strategy applied to **b** results in **{e,f}**.

Note that the same rule label can be used for naming different rules. In fact this is equivalent to design rules with different labels and a choice operator to link them. For instance in Example 3.13, we could have the label **r3** again instead of **r4** for the fourth rule. The strategy could then be defined as **dc(dk(r1), dk(r2), dk(r3))**.

Strategies concatenation

Two strategies are concatenated by applying the second strategy on all the results of the first one.

The concatenation operator denoted “;” builds the sequential composition of two strategies S_1 and S_2 . The strategy $S_1; S_2$ fails if S_1 fails, otherwise it returns all results (maybe none) of S_2 applied to the results of S_1 ;

Example 3.14 *In the same context as in the previous example (Example 3.13), the strategy:*

```

strategies for foo
  implicit
  [] conc =>
    dc(r1) ;
    dc(r5) ;
    dk(r3 , r4)
  end
end

```

when applied on the term **a** results in the application of first **r1**, then **r5**, and finally **r3** and **r4** in all possible ways, yielding the results **{e,f}**.

Strategy iterators

In order to allow for the automatic concatenation of the same strategy, ELAN offers four powerful iterators:

The strategy **iterate*** corresponds to applying zero, then one, then two, ... n times the strategy to the starting term, until the strategy fails. Thus **(iterate*(s))t** returns $\bigcup_{n=0}^{\infty} (s^n)t$. Notice that **iterate*** returns the results one by one, even when the iteration of the strategy never fails. This strategy cannot fail since it returns at least the initial term.

Its variant **iterate+** does not return the initial term but returns $\bigcup_{n=1}^{\infty} (s^n)t$. It can fail if s fails on the initial term.

The strategy **repeat** iterates the strategy until it fails and returns just the terms resulting from the last unfailling call of the strategy. The two variants are thus defined as:

(repeat*(s))t = $(s^n)t$ where $(s^{n+1})t$ fails, $n \geq 0$ and $s^0 = id$,
(repeat+(s))t = $(s^n)t$ where $(s^{n+1})t$ fails and $n > 0$.

Example 3.15 *To illustrate the basic behaviour of the elementary strategies, let us consider the following module:*

```

module stratFail
import global int bool;
end
sort foo ; end
operators
  global
  a : foo ;          b : foo ;
end
stratop
  global
  tryIt   : <foo -> foo>      bs ;
  repeatS : <foo -> foo>      bs ;
  repeatP : <foo -> foo>      bs ;
end
rules for foo
  global
  [a2b] a => b              end
end
strategies for foo
  implicit
  [] tryIt   => dc(a2b)          end
  [] repeatS => repeat*(dc(a2b)) end
  [] repeatP => repeat+(dc(a2b)) end
end
end

```

When applying the strategies on the terms **a** or **b**, we are obtaining the following results:

strategy	a	b
tryIt	{b}	$\emptyset$
repeatS	{b}	{b}
repeatP	{b}	$\emptyset$

In particular, the difference between the **repeat*** and **repeat+** is due to the fact that when $dc(a2b)$ is applied zero times, this is by definition the identity strategy, returning the initial term.

Example 3.16 The extraction of the elements of a list can be realized in the following way:

```

module iterRepeat

import global int bool;
end

sort elem nelist list; end

operators
  global
    @      :      ( int )      nelist ;
    @ . @   :      ( int nelist ) nelist ;
    element(@) :      ( nelist )      int ;
  end

stratop
  global
    all0      : <int -> int> bs ;
    allRepS   : <int -> int> bs ;
    allRepP   : <int -> int> bs ;
    allIter   : <int -> int> bs ;
  end

rules for int
  e : int;
  l : nelist;
  global
    [extract] element(e)      => e      end
    [extract] element(e.l)    => e      end
    [extract] element(e.l)    => element(l) end
  end

strategies for int
  implicit
    [] all0      => dk(extract)      end
    [] allRepS   => repeat* ( dk(extract) ) end
    [] allRepP   => repeat+ ( dk(extract) ) end
    [] allIter   => iterate* ( dk(extract) ) end
  end
end

```

We then obtain the following results for the different strategies applied on the terms 1 or $element(1.2.3)$ respectively:

strategy	1	$element(1.2.3)$
all0	$\emptyset$	{1, $element(2.3)$ }
allRepS	1	{1, 2, 3}
allRepP	$\emptyset$	{1, 2, 3}
allIter	1	{ $element(1.2.3)$, $element(2.3)$, $element(3)$, 1, 2, 3}

The normalize strategy

Since labelled rules are applied only at the top of the terms, it is useful to have at hand a way to apply a family of labelled rules everywhere in a term to get it normalized. This is provided in ELAN by the **normalize** strategy. This strategy takes a strategy and use it in order to normalize the term on which it is applied.

▷ There is no assumption of the way the rules are applied, e.g. neither innermost nor outermost.

Example 3.17 *A typical example of the use of **normalize** is the normalization process of the typed lambda calculus, whose main module is described as follows:*

```

module normLambda
import      global      int;
      local      bool eq[lterm];
end
sort lterm;
end
operators global
      @          : (int) lterm          ;
      (@ @)      : (lterm lterm) lterm  ;
      la @       : (lterm) lterm        ;
      local
      repl(@,@,@) : (int lterm lterm) lterm ;
      free(@,@)   : (int lterm) bool      ;
end
stratop global
      betaEta : <lterm -> lterm> bs ;
end
rules for bool
  v, w : int;
  M, N, Q : lterm;
global
  [] free(v,v) => true                      end
  [] free(v,(M N)) => free(v,M) or free(v,N) end
  [] free(v,la M) => free(v+1,M)             end
  [] free(v,M) => false                     end
end
rules for lterm
  v, w : int;
  M, N, Q : lterm;
global
  [] repl(v,v,M) => M                      end
  [] repl(v,w,M) => w                      end
  [] repl(v,(M N),Q) => (repl(v,M,Q) repl(v,N,Q)) end
  [] repl(v,la N,M) => la repl(v+1,N,M)    end
  [beta](la M N) => repl(1,M,N)            end
  [eta] (la M 1) => M if not free(1,M)      end
end
strategies for lterm
  implicit
  [] betaEta => normalise(first one(beta,eta)) end
end
end

```

The normalize strategy with trace

This new strategy (**normin**) normalizes terms with respect to a set of labelled rules in leftmost-innermost fashion. Moreover, a complete trace is generated (see option **-proofterm** of the compiler). This trace contains all informations about each rewrite step (*i.e.* the context, the

rule and the substitution) and allows the normalisation process to be exported to external systems. The strategy **normin** is used, for example in an ELAN based tactic for (simple or AC) rewriting in the Coq proof assistant available at [Ngu].

Identity and Failure

Two more constructions are available:

- **id** is the identity strategy that does nothing, and never fails.
- **fail** always fails and returns an empty set of results.

3.5.2 Defined strategies

The defined strategy language extends the elementary one by the possibility to define recursive strategies. The application of a defined strategy is itself performed by an ELAN program that defines the interpretation of the new constructions introduced by the user. All the modules defining the appropriate syntax and semantics are provided in the ELAN library.

The syntax of the defined strategy language is described in an ELAN module called **strsig[X,Y]** when strategies ranks contain sorts of the form $\langle X \rightarrow Y \rangle$ (strategies are non sort-preserving), and the module **strat[X]** when the strategies concern sorts of the form $\langle X \rightarrow X \rangle$ (strategies are sort-preserving). To use non sort-preserving strategies, one has to import the module **tcstrat[X,Y]** where **X** and **Y** are different sorts. These modules implement the interpreter of the defined strategy language using a set of labelled rewrite rules and a basic strategy **eval**. The syntax of defined strategies is given below:

```

<defined strategy rule> ::=
    | [ ] <strategy body> end
    | [. ] <strategy body> end
    | [ <rule label> ] <strategy body> end

<strategy body> ::= <Dstrategy term> ==> <Dstrategy term> [ { <if-where-choose> } + ]

```

<if-where-choose> is defined in Section 3.4.1.

<Dstrategy term> is a term built on the signature of strategy operators. It has a strategy sort. The different labels $\square$, $\cdot$ and ℓ for strategy rules correspond to different evaluation modes explained in Section 3.6.2.

Example 3.18 *An example of the use of these features is the following:*

```

module map[X]
import
  global
    strat[X] strat[list[X]] X list[X];
  end
stratop
  global
    map(@)      : (<X -> X>) <list[X] -> list[X]>;
    start @     : (int)    <list[X] -> list[X]>;
  end
rules for X

```

```

      x      : X;
      n      : int;
global
  [ass(n)] x => n          end
  [add(n)] x => x+n        end
  [sub(n)] x => x-n        end
end
strategies for list[X]
  s : <X -> X>;
  h, h1 : X;
  t, t1 : list[X];
  n : int;
explicit
  [.] [map(s)] nil => nil          end
  [.] [map(s)] h.t => h1.t1
                                where h1:=[s]h
                                where t1:=[map(s)]t          end
implicit
  [] start n      => map(dk(ass(n),dk(add(n),sub(n))))    end
end
end

```

Example 3.19 *The following module shows how to program loops with the defined strategy language.*

```

module loop
import global strat[int] int
      strconc[int,int,int];
end
stratop global
      loop          : <int -> int>;
end

rules for int
  x      : int;
local
  [sub1]      x => x-1 if x > 1          end
end

strategies for int
implicit
  [.] loop => dk(id, sub1;loop)          end
end
end

```

3.6 Evaluation mechanism

This section does not introduce any new syntactic material, but gives informal explanations on the operational semantics of the language.

3.6.1 Rewrite rules on terms

As we have seen, there are two kinds of rules: labelled and unlabelled. They define two classes of rules that are applied on the term to be evaluated in the following way:

- **Step 1** The current term is normalized using the unlabelled rules. This is done in order to perform functional evaluation and thus it is recommended to the user to provide a confluent and terminating unlabelled rewrite system to ensure termination and unicity of the result. This normalization process is built-in in the evaluation mechanism and consists in a leftmost innermost normalization. This yields always a single result.

- **Step 2** Then one tries to apply on the normalized term a labelled rule following the strategy described in the logic description. This leads to a (possibly empty) collection of terms. If this set is empty, then the evaluation backtracks to the last choice point; if it is not empty, then the evaluation goes on by setting a new choice point and evaluating one of the returned terms by going to **step 1**.

In a slightly more formal way, a rule

$$[\ell] \quad l \rightarrow r \quad s_1 \dots s_n$$

where the s_i are either **where** or **if** or **choose try** expressions, is applied on a term t by:

1. Matching l against t . This computes a multiset of substitutions (because of equational matching). If this set contains more than two elements, one is chosen and the other ones are stored for possible future backtracking. Let σ be the chosen substitution.
2. The evaluation goes on by evaluating the expressions $s_1, \dots, s_n$, one by one and in this order (i.e. from 1 to n).
3. If s_i is of the form **where** $x_i := (strat_i)t_i$, then one of the results (call it t'_i) of the application of the strategy $strat_i$ on the term t_i is chosen, and the substitution σ is extended by $x_i \mapsto t'_i$. The other results are stored for possible backtracking, and the evaluation goes on with s_{i+1} . If the strategy $strat_i$ fails on t_i , then we backtrack to the previous choice point.
4. If s_i is of the form **where** $p_i := (strat_i)t_i$, then one of the results (call it t'_i) of the application of the strategy $strat_i$ on the term t_i is chosen, and the substitution σ is extended by the matching substitution σ_i from p_i to t'_i . The other results are stored for possible backtracking, and the evaluation goes on with s_{i+1} . If the strategy $strat_i$ fails on t_i , then we backtrack to the previous choice point. Note that when p_i contains AC symbols AC-matching is called and there is an additional backtracking mechanism to find an adequate AC-match σ_i .
5. If s_i is of the form **if** c_i , then the term c_i is evaluated following the normalization strategy. If the result is the **bool** constant **true**, then one evaluates the next expression s_{i+1} , otherwise one backtracks to s_{i-1} .
6. If s_i is of the form

```

choose
  try  $s_1^1 \dots s_m^1$ 
  ...
  try  $s_1^j \dots s_k^j$ 
  ...
  try  $s_1^p \dots s_l^p$ 
end

```

then we can mimic the evaluation mechanism by replacing all occurrences of ℓ in strategy expressions by p rules $\ell_1, \dots, \ell_p$ (in this order) defined as follows for $j = 1, \dots, p$:

$$[\ell_j] \quad l \rightarrow r \quad s_1 \dots s_{i-1} \ s_1^j \dots s_k^j \ s_{i+1} \dots s_n$$

Since **choose try** is a recursive construction, this unfolding process is performed by a leftmost-innermost mechanism, until we get rule expressions with no occurrence of **choose try**.

▷ One should note that the term to be evaluated is *first* normalized by ELAN using the unlabelled rules and the resulting term is then reduced using the given strategy. The following example illustrates this behaviour.

Example 3.20

```

module normalizeFirst
  // The module identity is part of the standard library
  import
    local identity[term] ;
  end

  sort term ; end
  operators
    global
      a      : term;
      f(0)    : (term) term;
    end
  stratop
    global
      s1      : <term -> term> bs;
      s2      : <term -> term> bs;
    end
  rules for term
    x,y,z : term;
    global
      [r1] f(x) => x end
      []   f(x) => x end
    end
  strategies for term
    implicit
      [] s1 => dc(r1) end
      [] s2 => dc(r1,identity) end
    end
  end
end

```

Applying the strategy **s1** on the term **f(a)** gives no result since:

1. **f(a)** is normalized by the unlabelled rule into the term **a**,
2. then the normalization process tries to apply the labelled rule **r1** on **a** and fails. So no strategy applies! The set of results is empty.

When applying the strategy **s2** on **f(a)**, for the same reason the reduction process fails to apply the rule **r1** but the strategy **identity** can then be applied and the result is then **a** since the term is firstly normalized.

Example 3.21 Let us consider a brute force specification of the eight queens problem: How can one put eight queens on one chess-board in such a way that they do not interfere? In this example **p1** refers to the position of the queen in the first column, **p2** to the position of the second queen which should be in the second column and so on up to **p8**.

```

module queens
  import
    bool int list[int];
  end

  operators

```

```

global
  queens      : list[int];
local
  ok(@,@,@)   : ( int int list[int] ) bool;
end

stratop
  global
    queens : <list[int] -> list[int]> bs;
  local
    try_step : <int -> int> bs;

end

rules for list[int]
  p1,p2,p3,p4,p5,p6,p7,p8 : int;
  pp1,pp2,pp3,pp4,pp5,pp6,pp7,pp8 : list[int];
local
  [queensrule] queens      => p8.pp7
  // -- first: the position on the first row is choosen
  where p1:= (try_step) 0
  // -- second: the position on the second row is also choosen
  where p2:= (try_step) 0
  where pp1:=() p1.nil
  // -- third: the positions on the first and
  // second rows are checked to be compatible
  if ok(1,p2,pp1)
  // -- quarto: choose a position on the third row
  // and continue ...
  where p3:= (try_step) 0
  where pp2:=() p2.pp1
  if ok(1,p3,pp2)
  where p4:= (try_step) 0
  where pp3:=() p3.pp2
  if ok(1,p4,pp3)
  where p5:= (try_step) 0
  where pp4:=() p4.pp3
  if ok(1,p5,pp4)
  where p6:= (try_step) 0
  where pp5:=() p5.pp4
  if ok(1,p6,pp5)
  where p7:= (try_step) 0
  where pp6:=() p6.pp5
  if ok(1,p7,pp6)
  where p8:= (try_step) 0
  where pp7:=() p7.pp6
  if ok(1,p8,pp7)

end
end

rules for bool
  p,d,diff : int;
  l : list[int];
global
  [] ok(diff,d,nil)      => true      end
  [] ok(diff,d,p.l)      => false      if d == p      end
  [] ok(diff,d,p.l)      => false      if d-p == diff  end
  [] ok(diff,d,p.l)      => false      if p-d == diff  end
  [] ok(diff,d,p.l)      => ok(diff+1,d,l)      end
end

rules for int
  x:int;
local
  [tryrule] x      => 1      end
  [tryrule] x      => 2      end
  [tryrule] x      => 3      end
  [tryrule] x      => 4      end
  [tryrule] x      => 5      end

```

```

    [tryrule] x      => 6      end
    [tryrule] x      => 7      end
    [tryrule] x      => 8      end
end

strategies for int
  implicit
    [] try_step => dk(tryrule) end
  end

strategies for list[int]
  implicit

    [] queens => dc(queensrule) end
  end
end

```

*Note that the strategy above returns only one solution. If it is changed to a **dk**, we get all possibilities.*

*Note also the way all the numbers between 1 and 8 are enumerated in ELAN using a **dk** strategy (see **tryrule**).*

3.6.2 Rewrite rules on strategies

Elementary strategies are defined by rules of the form `[] c => strat` where `c` is a constant of sort `<s -> s'>` and `strat` a term built on elementary strategy constructors. Such rules are always unlabelled. Application of such a strategy `c` on a term `t`, denoted `(c)t` is performed by a C function generated by ELAN.

Defined strategies are like ordinary terms except that they have a functional sort of the form `<s -> s'>`. To understand how rules are applied on these strategy terms, it is helpful to distinguish two kinds of rule purposes: either the rule defines the result of the application of a strategy to a term (and involves implicitly or explicitly the application operator `[@]@`), or the rule defines a computation on strategy terms.

- Consider first the case of a rule which defines the result of the application of a strategy to a term.
 - When the rule is labelled by `[.]`, it is used by the strategy interpreter (i.e. the ELAN program given in the module `strat[X]` for sort preserving strategies or `strat[X,Y]` for others). For that, the rule is automatically transformed into a rule with the label `DSTR` and is applied with the default strategy `eval` of the strategy interpreter.
 - When the rule is labelled by `[ℓ]`, it is used by the ELAN interpreter as any other labelled rule governed by a strategy. So the user has to provide a strategy involving `ℓ`.
- Rules that define a computation on strategy terms are evaluated exactly like rules on (ordinary) terms:
 - If the rule is unlabelled, the leftmost innermost predefined strategy of the interpreter is applied.
 - If it is labelled by `[ℓ]`, the evaluation process needs a user-defined strategy involving `ℓ`.

Example 3.22 We define here a small strategy library composed of several defined strategies that provide primitives for depth-first search in a computation tree. This strategy library is used to build a strategy that finds out a path (or, all paths) towards an exit of a labyrinth.

The strategy library supports four primitives for depth-first search. The current situation of the partially discovered search tree is represented by a path from the initial state (the root of the tree) to the current state (the left-most leaf). It also remembers all possible alternatives (or, choice points) met along this path. Each step of the path is represented by a list of states, where its head is the currently chosen state, and the rest represents all non-explored alternative states. Each step of the path is a list of states, thus, the whole path could be naturally represented as `list[list[state]]`. This sort also describes the current situation during the depth-first search in this tree.

Four primitive strategies are defined over the sort `list[list[state]]`.

- **call**(*S*) applies the strategy *S* on the current state, where *S* is of sort $\langle \text{state} \rightarrow \text{state} \rangle$. All possible results of this application, if there are any, are grouped together into a new step (level) attached to the current path.
- **next** throws away the current state and continues searching with the next possibility of the previous step (the last choice-point).
- **exit** leaves the current state ignoring all alternatives and it returns the control to the previous step of the current path.
- **cut** eliminates all alternative states of the last step.

```

module space

import global      list[state]
                  list[list[state]]
                  eq[state] eq[list[state]]
                  strapply[state]
                  bool
                  strat[state]
                  strat[list[list[state]]];

end
sort state; end
stratop global
  next          : <list[list[state]] -> list[list[state]]>;
  exit          : <list[list[state]] -> list[list[state]]>;
  cut           : <list[list[state]] -> list[list[state]]>;
  call(@)       : (<state -> state>) <list[list[state]] -> list[list[state]]>;
  loop          : <list[list[state]] -> list[list[state]]>;
  narrow        : <list[list[state]] -> list[list[state]]>;
  local
    occurs @ in @ : (state list[list[state]]) bool;
  end
rules for bool
  state1, state2 : state;
  level          : list[state];
  space          : list[list[state]];
global
  [] occurs state1 in nil => false
  [] occurs state1 in state2.level.space => true if state1 == state2
  [] occurs state1 in state2.level.space => occurs state1 in space
end

strategies for list[list[state]]
  state : state;
  s      : <state -> state>;
  level  : list[state];

```

```

        space                : list[list[state]];
        new_level             : list[state];
        new_space              : list[list[state]];
explicit
  [.] [next]state.level.space => level.space          end
  [.] [exit]state.level.space => space                end
  [.] [cut]state.level.space  => state.nil.space      end
  [.] [call(s)]state.level.space
        => new_level.state.level.space
        where new_level:=()setof(s,state)
        if new_level != nil      end
  [.] [loop]state.level.space => state.level.space if occurs state in space end
  [.] [narrow]state.level.space => state.nil.new_space
        where new_space:=[narrow]space      end
  [.] [narrow]nil => nil                      end
end
end

```

The variable *state* represents the current state, the variable *level* represents all alternatives of the current state, i.e. *state.level : list[state]* is one step of the path, where the others steps are recorded in the variable *space*.

The only non-trivial strategy among the four primitives is the strategy *call(S)*, which uses the function symbol *set_of* to collect all results of an application of a strategy *S*.

In order to design a strategy helping to find out an exit from a labyrinth, we split the problem into a part dependent on the labyrinth and an independent one. The dependent part contains a specification of a state, which is a pair of coordinates, and a definition of four basic moves in the labyrinth. They are realized as state transforming strategies *left*, *right*, *up*, *down* of sort $\langle \text{state} \rightarrow \text{state} \rangle$ such that these strategies fail, if some movement in a given state is not possible (because of a wall). All exits of the labyrinth are specified by a strategy *exitable*, which fails, if the current state has ‘no exit doors’. Otherwise, it is equivalent to the identity strategy.

```

module robot

import global
    state space
    strat[state] strat[list[list[state]]]
    int bool labyrinth
    strconc[list[list[state]],list[list[state]],list[list[state]]];
end
stratop global
  up          : <state -> state>;
  down        : <state -> state>;
  left        : <state -> state>;
  right       : <state -> state>;
  exitable    : <state -> state>;
  search      : <list[list[state]] -> list[list[state]]>;
  allsearch   : <list[list[state]] -> list[list[state]]>;
  local
    moves     : <state -> state>;
  end

rules for state
  x,y,dx,dy,dir : int;
local
  [move(dx,dy,dir)]
    state(x,y) => state(x+dx,y+dy)
    if room(x,y) & dir != 0
  end
end

strategies for state
implicit
  [.] up => move( 1, 0, U)          end
  [.] down => move(-1, 0, D)        end
  [.] left => move( 0,-1, L)        end
  [.] right => move( 0, 1, R)       end

```

```

[.] exitable=>move(0,0,E)                                end
[.] moves =>      dk(right, dk(down, dk(left, up)))      end
end

strategies for list[list[state]]
implicit
[.] search => if call(exitable) then
              narrow
            orelse if loop then
                  next;search
            orelse
              dc(call(moves);search, next;search)
            fi
          fi
[.] allsearch => dk(call(exitable);narrow,
                  if loop then next
                  orelse
                    dk(call(moves), next)
                  fi;
                  allsearch)
end
end

```

The independent part of the strategy consists of a definition of the strategy *search* of sort $\langle \text{list}[\text{list}[\text{state}]] \mapsto \text{list}[\text{list}[\text{state}]] \rangle$ and an auxiliary strategy *moves* of sort $\langle \text{state} \mapsto \text{state} \rangle$. The strategy *search* succeeds on an exitable state (a room with exit doors). Otherwise, if the current path contains a loop, it backtracks and takes the next possible state to go on. If it does not contain a loop, it either moves to the neighbour state and goes on searching, or, if no exit has been found, it ignores the current choice, and backtracks to the next alternative.

3.7 Modules

The modularity constructions in the current version of ELAN are as simple as possible and the semantics of parametrisation as well as importation is textual expansion.

The top-level information that should be provided by the super-user is the description of the logic (s)he wants to make available. This is achieved in ELAN by using two kinds of modules. The first one describes the top level of the logic and are presented in files having the ‘.lgi’ extension, the second ones contain all subsequently needed informations. They are called element modules and they are presented in files having the extension ‘.eln’. Element modules have been described in Section 3.2.

3.7.1 Visibility rules

The visibility rules in ELAN are the following:

- Only sorts are global. This means that they can be used in any module without any importation command. As a consequence, their name is unique in the ELAN universe built for a run.
- Operators, rewrite rules and strategies can on the contrary be local or global. A local operator will be known only in its own module (i.e. the module where it is defined). If an operator is declared to be global, then it is known in all modules importing the module where it is declared. This can be slightly modified, when qualifying the importation command by local or global. By default an importation without local or global specification is assumed to be local.

- A special case is for aliases. The visibility rules for them are the same as for operators. The only interesting case is for a module that locally imports a signature but exports the alias definition. In this case, only the alias name of the operator will be known outside and not its original name: this is quite useful for renaming purposes.

— Release Note: *The main change concerning the visibility rules is that now only sorts are always global. See section 3.9 for a summary of all syntax modifications.*

3.7.2 Built-in modules

For convenience and efficiency several modules are provided as built-ins. See section 5.2 for a full description of the current available ones.

3.7.3 Parameterised modules

Parameterised modules can be defined in ELAN as follows:

$$\begin{aligned} \langle \text{formal module name} \rangle &::= \langle \text{identifier} \rangle \\ &| \langle \text{identifier} \rangle [\langle \text{identifier} \rangle \{ , \langle \text{identifier} \rangle \}^*] \end{aligned}$$

The module instantiation by its actual parameters is currently done by the syntactical replacement of the formal arguments by the actual ones.

Example 3.23 *The classical example is the one of parameterised lists:*

```

module simpleList[X]
  sort X simpleList[X]; end
  operators
    global
      nil          : simpleList[X];
      @ . @        : ( X simpleList[X] ) simpleList[X];
      append(@,@)  : ( simpleList[X] simpleList[X] ) simpleList[X];
    end
  rules for simpleList[X]
    e      : X;
    l,l1,l2 : simpleList[X];
    global
      [] append(nil, l)      => l          end
      [] append(e.l1, l2)    => e . append(l1,l2) end
    end
  end
end

```

and we get list of integers by instantiating X by int and importing int as in:

```

LPL simpleList description
  query of sort      simpleList[int]
  result of sort     simpleList[int]
  import            int simpleList[int]
  start with        ()query
end

```

3.7.4 LGI modules

The top level description of the logic consists of:

1. the description of the syntax that the user is allowed to use in order to give his specifications.
2. the description of a term, called the query, that will be reduced in the rewrite logic defined by the (super) user.

The syntax is the following:

```

<lpl description> ::= LPL <identifier> description
                    [ <specification description> ]
                    query
                      of sort <sort name>
                      result of sort <sort name>
                      import { <instantiated module name> } +
                      [ check with <boolean term> ]
                      start with <start term>
                    end

<specification description> ::= specification description
                                { <specification part description> } +
                                end

<specification part description> ::= part <identifier>
                                     of sort <sort name>
                                     import { <instantiated module name> } +
                                     [ check with <boolean term> ]

<start term> ::= ( [ <Estrategy name> ] ) <query term>

```

The syntax of *<query term>* is built over the user-defined signature enriched by the constant **query**. The sort of **query** is given by the query declaration above.

Example 3.24 *If no specification is needed, the specification description part is skipped, as in the following logic description used for running Example 3.13:*

```

LPL testStrat description

query
  result      of sort foo
  import      testStrat
  start with (strat) query

end

```

Example 3.25 *Here is a simple example of the top level description of how unification can be implemented:*

```

LPL unification description
    // This part describes the syntax in which the user should
    // complete the definition of the logic: i.e. how (s)he gives
    // the specification.
specification description

    part Vars          of sort list[identifier]
                        import sigSyntax
    part Ops            of sort list[pair[identifier,int]]
                        import sigSyntax
end
    // The following part describes the way the user should gives the query
    // and how it will be evaluated
query                of sort constraint
                    result of sort constraint
                    import unification[Vars,Ops]
                        constraint[Ops,Vars]
                        termCommons[Ops,Vars]
                    start with (unify) query
end

```

In this unification logic, the user should provide the specification information as follows:

```

specification simple sig
    Vars  x y z u v w
    Ops   a:0 b:0 c:0 f:2 g:1
end of specification

```

*The user should use the (super-user defined) keywords **Vars** and **Ops** to define respectively the variables and the operators, namely **a** of arity 0, **f** of arity 2, etc...*

Semantics

When parsing a specification, ELAN generates new modules that complete the definition of the rewrite theory to be used during the evaluation of the query. For instance, in the case of the previous example, these new modules are:

```

module Vars
import anyIdentifier sigSyntax; end
operators
    global
        Vars : list[identifier];
    end
rules for list[identifier]
    global
        [] Vars => x y z ;
    end
end

and

module Ops
import anyIdentifier identifier sigSyntax list[pair[identifier,int]]; end
operators
    global
    Ops : list[pair[identifier,int]]; end

```

```

rules for list[pair[identifier,int]]
global
[] Ops => a:0 b:0 c:0 f:2 g:1 end
end

end

```

When a user provides a specification to an ELAN logic, this simply provides more information about the context in which the user wants to work, like the name of function symbols or basic axioms to use.

3.8 Pre-processing

Another original feature proposed by ELAN is the use of a pre-processing phase that allows to describe the logic to be encoded in an easier way.

As described in Figure 1.1, the pre-processor is performing textual replacements starting from informations given by the super-user in the modules, and by the user in the specification file. The pre-processing phase in ELAN can just be thought of as a macro expansion mechanism which extends the parameterization process described before.

In order to express the syntax of the pre-processor construction, we need the notion of *constant expression* and of *text* defined as follows:

$$\begin{aligned}
 \langle \text{constant expression} \rangle & ::= \langle \text{number} \rangle \\
 & \quad | \quad (\langle \text{subexpression} \rangle) \\
 \\
 \langle \text{subexpression} \rangle & ::= \langle \text{number} \rangle \\
 & \quad | \quad \langle \text{subexpression} \rangle + \langle \text{subexpression} \rangle \\
 & \quad | \quad \langle \text{subexpression} \rangle - \langle \text{subexpression} \rangle \\
 & \quad | \quad \langle \text{subexpression} \rangle * \langle \text{subexpression} \rangle \\
 & \quad | \quad \langle \text{subexpression} \rangle / \langle \text{subexpression} \rangle \\
 & \quad | \quad \langle \text{subexpression} \rangle \% \langle \text{subexpression} \rangle \\
 & \quad | \quad (\langle \text{subexpression} \rangle) \\
 \\
 \langle \text{lexem} \rangle & ::= \langle \text{identifier} \rangle \\
 & \quad | \quad \langle \text{number} \rangle \\
 & \quad | \quad \langle \text{char} \rangle \\
 \\
 \langle \text{text} \rangle & ::= \{ \langle \text{lexem} \rangle \}^+
 \end{aligned}$$

3.8.1 Simple duplication

A first feature is to allow simple duplications of a part of text. The syntax is:

$$\begin{aligned}
 \langle \text{simple repetition} \rangle & ::= \langle \text{lexem} \rangle^{\sim} \langle \text{constant expression} \rangle \\
 & \quad | \quad \{ \langle \text{text} \rangle \}^{\sim} \langle \text{constant expression} \rangle
 \end{aligned}$$

Example 3.26 The text “a{,b}~3” is processed into “a, b, b, b”.

3.8.2 Duplication with argument

It is often necessary to allow description of objects like $f(t_1, \dots, t_5)$. This is possible using the syntax:

$$\begin{aligned} \langle \text{arg.repetition} \rangle &::= \{ \langle \text{text} \rangle \}_- \langle \text{identifier} \rangle \\ &= \\ &\quad \langle \text{constant expression} \rangle \dots \langle \text{constant expression} \rangle \end{aligned}$$

Example 3.27 The text “ $P \{ \& s_I=t_I \}_I=1\dots 3$ ” is processed into “ $P \& s_1=t_1 \& s_2=t_2 \& s_3=t_3$ ”.

A special form of this duplication with arguments is the explicit construction of a list of indexed identifiers allowed using the syntax:

$$\langle \text{arg.repetition} \rangle ::= \langle \text{identifier} \rangle _ \langle \text{number} \rangle \langle \text{char} \rangle \dots \langle \text{char} \rangle \langle \text{identifier} \rangle _ \langle \text{number} \rangle$$

Example 3.28 The text “ $t_1, \dots, t_5$ ” is processed into “ $t_1 , t_2 , t_3 , t_4 , t_5$ ”.

3.8.3 Enumeration using FOR EACH

This construction allows to make the link between the specification given by the user and the logic described by the super-user. A natural motivation for this construction is given by the “standard” way inference rules are used. For example when describing how unification works, the following transformation rule is given:

Decompose $P \wedge f(s_1, \dots, s_n) =^? f(t_1, \dots, t_n) \rightarrow P \wedge s_1 =^? t_1 \wedge \dots \wedge s_n =^? t_n$

It is generic in the sense that the operator f is unspecified. It can be a $+$ of arity 2, or a *if@then@else@* of arity 3, or just a constant. We do not want, when specifying how the logic works, to give only the specific cases, we need to be as generic as possible.

ELAN provides via the FOR EACH construction of the pre-processor, a way to be generic. The syntax is the following:

$$\begin{aligned} \langle \text{for each const.} \rangle &::= \text{FOR EACH } \langle \text{vardecl} \rangle \text{ SUCH THAT } \langle \text{varaffect} \rangle : \{ \\ &\quad \langle \text{text} \rangle \\ &\quad \} \\ \langle \text{vardecl} \rangle &::= \langle \text{varname} \rangle \{ , \langle \text{varname} \rangle \}^* : \langle \text{sort name} \rangle \\ &\quad | \quad \langle \text{vardecl} \rangle ; \langle \text{vardecl} \rangle \\ \langle \text{varaffect} \rangle &::= \langle \text{varname} \rangle := ([\langle \text{strategy name} \rangle]) \langle \text{term} \rangle \\ &\quad | \quad \langle \text{varaffect} \rangle \text{ AND } \langle \text{varaffect} \rangle \\ &\quad | \quad \langle \text{varaffect} \rangle \text{ ANDIF } \langle \text{boolean term} \rangle \end{aligned}$$

▷ Since the symbols $\{, \}, \sim$ are reserved for pre-processor use, it is not possible to use them for any other purpose, even quoted.

▷ The pre-processor uses the character ‘ ’ (underline) for assembling identifiers. For example, the text ‘a f 1_ x b’ is pre-processed into the sequence of three identifiers a, f_1_x and b. The character ‘ ’ is thus part of an identifier even if there is a space between it and the rest of the string. It is thus better for the user not to use the symbol, except for building identifiers of course.

Example 3.29 *The rule **Decompose** that we mentioned at the beginning of this section can be expressed in the following way:*

```
FOR EACH SS:pair[identifier,int]; F:identifier; N:int
SUCH THAT SS:=(listExtract) elem(Fss) AND F:=( )first(SS) AND N:=( )second(SS) :{
  rules for constraint
    s_1,...,s_N:term; t_1,...,t_N:term;
  local
    [decompose] P & F(s_1,...,s_N)=F(t_1,...,t_N) => P { & s_I=t_I }_I=1...N
  end
end
}
```

*If the specification given by the user consists in the operator symbols **f** and **g** of respective arities 2 and 1, then the pre-processor expands the previous **FOR EACH** construction into:*

```
rules for constraint
  s_1, s_2, t_1, t_2 : term;
  local
    [decompose] P & f(s_1,s_2)=f(t_1,t_2) => P & s_1=t_1 & s_2=t_2 end
    [decompose] P & g(s_1)=g(t_1) => P & s_1=t_1 end
  end
```

3.9 Differences with previous version of the language

The language has evolved a lot between ELAN version 1.17 and ELAN version 3.0. The changes concern the syntax of:

- modules,
- rules and
- strategies.

The main changes are emphasized in the corresponding sections of the language description, and the main concern has been to uniformize the language constructions.

The transformations needed to change from the syntax of version 1.17 to the current one (i.e. later than 2.00) are summarized below.

Concerning the declaration parts:

- `import ...` becomes `import ... end`
- `sort ...` becomes `sort ... end`
- `op ... endop` becomes `operators ... end`

Now, there is only one construction to define rules, you have to replace:

- `rule for ...` by `rules for ...`
- `rule <name> for ...` by `rules for ... [name] ...`
- suppress `declare` keyword
- body or bodies by `global` or `local`

- end of rule, end of rules by end

The rule labels accept now arguments that should be variables involved in the rule.

Definition of strategies looks like rules definitions, you have to replace:

- strategy ... by strategies for ... global|local [name] ...
- iterate ... enditerate by iterate*(...)
- while ... endwhile by repeat*(...)
- repeat ... endrepeat by repeat*(...)
- dont know choose(...) by dk(...)
- dont care choose(...) by dc(...)
- end of strategy by end
- end of module by end

Local affectations and conditions are now evaluated from top to bottom. The order of presentation has been reversed, so that the *old* syntax:

```
[] 1 => r
    if c2
    where v2:=(...) ...
    where v1:=(...) ...
    if c1
```

becomes in the *new* syntax:

```
[] 1 => r
    if c1
    where v1:=(...) ...
    where v2:=(...) ...
    if c2
```

Chapter 4

ELAN: the system

This section is devoted to the user environment of the ELAN language, and is partly borrowed from [BKK⁺98b].

4.1 Global architecture

The ELAN environment consists of several components, as depicted in Figure 4.1. The preprocessor expands a few concise constructions allowed in the language. The parser checks the syntax of programs and verifies that terms are syntactically well-formed. The interpreter is an interactive tool allowing the user to check that the results he expects are indeed obtained. Initially, the preprocessor and the parser were integrated in the interpreter and so it was not possible to call them as stand-alone processes. The compiler transforms specifications into independent executable C code (Section 4.5). It is a stand-alone tool without any preprocessing and parsing facilities. The preprocessing and parsing phases are provided by the interpreter, which communicates the result of these processes to the compiler via a data exchange format called REF. Section 4.6 describes some tools to translate ELAN to REF and vice versa. Especially, the translation from ELAN to REF performed by the `query2ref` tool relies mainly on reusing the parser initially integrated in the interpreter.

4.2 The preprocessor

The ELAN syntax provides a few fancy constructions to perform textual replacements. So the preprocessor may be used to automatically generate parts of specifications used to analyse the rest of a program. It should be emphasised that there is strong interaction between the parser, the preprocessor and the interpreter. See Section 3.8 for a description of the preprocessor functionality.

4.3 The parser

The ELAN parser is based on Earley's one [Ear70] and is using sort informations in order to determine as soon as possible in the analysis the right choice. This explains why in the current version, some construction of the language require to give the sort information together with the term.

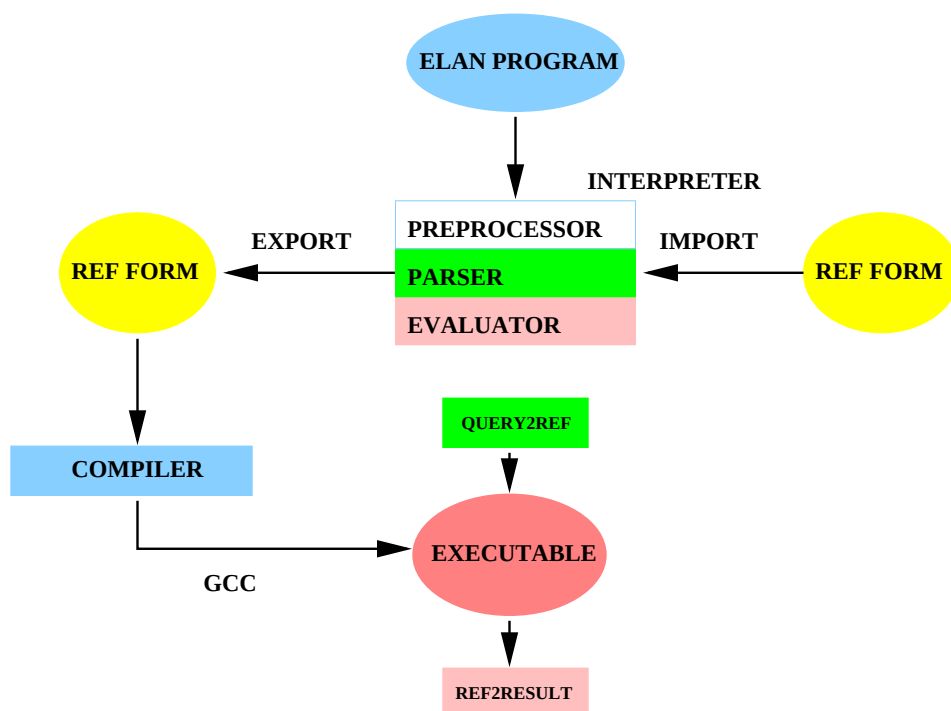


Figure 4.1: Global architecture of the ELAN environment

4.4 The interpreter

The interpreter takes a well-formed program and a well-formed query (both checked by the parser) and applies the rules and strategies defined in the program to the query. In order to find which rules can apply, the selection is guided by the top symbol of the rules: only those rules whose left-hand side has the same top symbol as the term to be reduced are selected. They are then tried in the order given in the program. Another kind of choices arbitrarily made by the interpreter is for the strategy $\mathbf{dc}(S_1, \dots, S_n)$ that should select randomly a non-failing strategy among $S_1, \dots, S_n$. In practice, the interpreter selects the first one, so implements $\mathbf{dc}$ and $\mathbf{first}$ in the same way. Note however that there exists a version of ELAN which concurrently executes the n strategies and selects the first one which terminates without failure [BC98].

Once a set of rules is selected, a many-to-one matching algorithm is applied. When associative and commutative (*AC* for short) operators are involved, an external one-to-one *AC*-matching algorithm described in [Eke95] is called. This algorithm is not fully integrated in the interpreter, so data structure conversions are required and lower the efficiency of the *AC*-matching, already quite complex. Once a match is found, local evaluations are performed and if all succeed, the result term is built, taking advantage of the right-hand side of the rule and of term sharing.

4.5 The compiler

The ELAN compiler transforms a logic description into an executable binary file. The compiler (for detailed description, see [Vit96, MK98]) produces an efficient C code, which is later compiled by the `cc` or the `gnu gcc` compiler. The executable binary code is dependent on the particular architecture, because a small part of the ELAN library performing the basic non-deterministic operations is written in assembly language [Mor98]. Up to now, this is the only reason that

makes the compiler available only on the architectures DEC-ALPHA, SUN4 and Intel-PC.

4.5.1 Non-linear rules

The ELAN compiler does not support non-left-linear rules (a rule is called non-left-linear if a variable occurs more than once in the left-hand side). The many-to-one matching algorithm only compiles linear rules. The ELAN parser automatically transforms non-left-linear rules into left-linear conditional rules by renaming variables that occurs more than once and adding some conditions to ensure their equality.

For example, the rule $f(x,x) \Rightarrow g(x)$ is transformed into $f(x,y) \Rightarrow g(x) \text{ if } x=y$.

4.5.2 Built-ins

To be more efficient, builtin data type such as *builtinInt* and *identifier* are compiled in a particular way.

The main drawback of this approach is that only completely defined functions can be defined on builtin data type.

Let $f(@) : (\text{builtinInt}) \text{ builtinInt}$ be an operator. For each integer n , $f(n)$ must be reducible to an integer. Suppose defined the rule $[] f(n) \Rightarrow n-1 \text{ if } n > 1$, the term $f(0)$ can not be reduced to a builtin integer. In this case, behaviour is undefined. This explains why *builtinInt* are never used directly: it is recommended to use the sort *int* which is no longer a builtin data type.

4.5.3 Input/Output facilities

Input/Output facilities are now fully implemented in the compiler. Their syntax is described in the module **IO.eln** of the standard library.

As in the version V3.4, it is possible, in the generated program, to enter a query term in the ELAN syntax, just like in the interpreter. In the same way, result terms computed by the generated program are now pretty-printed in the ELAN syntax. Therefore, it is no more necessary to give a fixed ground term in the *.lgi* file, and *.lgi* files involving the **query** keyword can now be compiled. In the current version, the query is parsed dynamically in the generated program by reusing a subpart of the parser integrated to the interpreter.

The generated program can be executed with several switches used to specify the different possible formats of input (query) and output (result) terms:

Input switches

- **-REFInput**: the query term must be in the REF format (see Section 4.6).
- **-noInput**: the query ground term is supposed to be located in the *.lgi* file, where it replaces the **query** keyword.

Output switches

- **-REFOutput**: result terms will be given in the REF format (see Section 4.6).
- **-noOutput**: results are computed but not written out. The interest of this switch is rather limited, but it can be useful for people who only want the statistics of the computation, like the number of rewrite steps per seconds.

- **-internalOutput**: result terms are given by using a prefix notation. This is the original syntax used in the first version of the compiler.

The generated program, called by default *a.out*, can be executed with some other switches not necessarily related to Input/Output, like **a.out -quiet** or **a.out -debug**.

At this point, it is still possible to change the content of the **start with** instruction declared in the *.lgi* file. The switch **-strategy <strategy name> : <sort name>** will apply the strategy *<strategy name>* on a query term of sort *<sort name>*.

Similarly, the switch **-sort <sort name>** will apply the empty strategy on a query term *<sort name>*. Note that **-strategy** and **-sort** are mutually exclusive.

▷ The list of available sorts (resp. strategy names) can be found in the *.ref* file.

4.6 The REF format

The Reduced ELAN Format, REF for short, is a term-like representation of ELAN programs introduced both for the interconnection of software components involved in the ELAN system, and for combining ELAN programs. This format is also of greatest interest for the implementation of reflection techniques in ELAN. A detailed description of its applications can be found in [BKK⁺98b].

The REF format has been initially developed to reuse the powerful Earley's parser integrated in the interpreter. By loading with the interpreter an ELAN program possibly in mixfix notation, we can get a REF program which can be viewed as the external representation of the interpreter working memory. REF is easier to parse than ELAN since the REF grammar is simply LALR. The ELAN interpreter generates a REF file *prog.ref* thanks to the option **--export**:

```
elan --export prog.ref prog.lgi [prog.spc]
```

The REF program contained in *prog.ref* can be directly executed by the interpreter as follows:

```
elan --import prog.ref
```

Then, the behavior of the interpreter is exactly the one expected with the command:

```
elan prog.lgi [prog.spc]
```

but without the time-consuming preprocessing and parsing phases.

The core of the ELAN compiler, called REM for “Reduced ELAN Machine” and implemented in Java, only knows the REF syntax. This explains why we need a script *elanc* to first call the interpreter in order to obtain a REF file, and second to execute REM with this REF file as input.

Then, a compiled C program, say *a.out*, is generated by REM. This program can be executed with different options like **a.out -REFInput**, **a.out -REFOutput** or even better **a.out -REFInput -REFOutput**:

- The option **-REFInput** reads a new query in REF syntax to replace the ground query occurring initially in *prog.lgi*.
- The option **-REFOutput** writes all result terms computed by *a.out* in REF syntax, instead of the ELAN syntax used by default.

So, we still have to convert an ELAN query to a REF query, and conversely to retrieve the ELAN syntax of result terms written in REF syntax.

For this purpose, one can use another tool to read a query and to pretty-print result terms of the *a.out* program, both in ELAN syntax, similarly to the interpreter. This tool is called *prettyIO* and may be used as follows:

```
prettyIO a.out prog.ref
```

Note that *prog.ref* is needed to find the ELAN syntax of functions implemented by *a.out*, and so to encode a query term provided by the user in ELAN syntax, and to decode terms written in REF syntax by *a.out -REFInput -REFOutput*. The executable **prettyIO** is implemented as a script calling some basic tools:

- **query2ref prog.ref** reads repeatedly (from **stdin**) a sort, and parses a term of this sort written in ELAN syntax. The result (written to **stdout**) is the corresponding term in REF syntax. By using the option **--query** as follows:

```
query2ref prog.ref --query
```

we only parse terms of the sort of the query, as indicated in *prog.lgi* (this information also occurs in *prog.ref*).

- **ref2result prog.ref** reads repeatedly (from **stdin**) terms encoded in REF syntax and writes (to **stdout**) the related terms in ELAN syntax, by using the signature occurring in *prog.ref*.

The script **prettyIO** is located in the directory **bin/** like **elanc**, and the compiled C/C++ programs **query2ref** and **ref2result** are located in the subdirectory of **bin/** related to the current system architecture, like the interpreter **elan**.

▷ The user should be aware that **prettyIO** is much less interesting in the current version, since it is now possible to parse a query term and to pretty-print all results by simply using the generated program *a.out* itself.

Chapter 5

The standard library and the built-ins

This chapter presents the main features of the ELAN standard library. Remember that the path to this library is specified in the system variable ELANLIB whose default value is set to the subdirectory `elan_library/elanlib` of the source directory. ELAN can be used without any reference to this library, *except* for what concerns the use of the built-in objects which are documented in the second part of this chapter.

This library has been designed to be small and as efficient as possible. In particular *no* AC operators is used. The resulting code is more efficient, at the price of sometimes heavier descriptions.

5.1 The ELAN standard library

The standard library is fully documented in [BCD⁺98] to which we refer for details. We give below the list of files defined in the standard library. Some of them define built-in sorts and operators and in this case, they also appear in the next section (section 5.2).

5.1.1 common

In this category, the most common abstract data types are defined.

<code>anyIdentifier.eln</code>	<code>eq.eln</code>
<code>anyInteger.eln</code>	<code>identity.eln</code>
<code>array.eln</code>	<code>int.eln</code>
<code>arrayLIN.eln</code>	<code>integerConstants.eln</code>
<code>arrayLOG.eln</code>	<code>IO.eln</code>
<code>bool.eln</code>	<code>list.eln</code>
<code>builtinArray</code>	<code>occur.eln</code>
<code>builtinHashTree</code>	<code>pair.eln</code>
<code>builtinInt.eln</code>	<code>prompt.eln</code>
<code>builtinIO.eln</code>	<code>replace.eln</code>
<code>builtinSidio.eln</code>	<code>stdio.eln</code>
<code>builtinString.eln</code>	<code>string.eln</code>
<code>builtinSyntacticMatching.eln</code>	<code>strlist.eln</code>
<code>cmp.eln</code>	<code>tuple.eln</code>
<code>Query.eln</code>	

5.1.2 ref

All the modules needed to deal with the Reduced ELAN Format.

```
Meta_Apply.eln ref_modules.eln ref_terms.eln
REF.eln ref_read.eln ref_unify.eln
ref_rules.eln ref_unify_builtin.eln
ref2string.eln ref_sorts.eln ref_write.eln
ref2term.eln ref_strategies.eln refstring2string.eln
ref_idents.eln ref_tables.eln string2refterm.eln
```

5.1.3 noquote

These modules describe a possible syntax for the user specifications. More complicated syntax (e.g. mixfix) can also be defined.

```
applySubstOn.eln hornClauseSyntax.eln syntacticUnification.eln
atom.eln ident.eln termCommons.eln
atomP.eln identifier.eln sigSyntax.eln
eqSystem.eln substitution.eln
```

5.1.4 strategy

These modules are used for the definition and evaluation of defined strategies.

```
Any.eln booleg.eln meta.eln strsig.eln
any.eln Meta_apply.eln strapply.eln symbol.eln
Meta_capply.eln commit.eln strat.eln tcstrat.eln
Meta_strat.eln strcapply.eln loops.eln strconc.eln
```

5.2 The built-ins

There are two types of built-in symbols:

- **The built-in constructors**, which are non-reducible symbols, built into the ELAN system. Terms built over them are either constructed, or interpreted by the ELAN system, e.g. the symbol `Error` defined in the module `.../common/builtinIO.eln` as follows:

```
Error(@) : (builtinInt) X    code 122;
```

which is used to report various errors of the I/O sub-system written in C++.

- **The built-in functions**, which represent symbols with pre-defined semantics, in general, not [easily] specifiable in ELAN itself. The symbol `open` in the module `.../common/builtinStdio.eln` defined as follows:

```
open_builtinStdio(@,@)      : (builtinString builtinString) builtinInt code 116;
```

can be taken as an example. In this case, the declaration (profile) of the symbol `open` represents an interface between the ELAN specification and its built-in semantics. In this case, the code “116” identifies its semantics.

If there is an overloaded built-in symbol whose exact semantics depends on its profile, a procedure implementing the built-in symbol should know, which instance of the built-in symbol is executed. The overloaded symbol `read_builtinIO` is an example of this problem:

```
read_builtinIO(@,@)      : (builtinInt X) X    pri 200 code 120;
```

When the built-in symbol `read_builtinIO` is used for several sorts X , there exist several built-in symbols `read_builtinIO` in the user's specification. All of them are linked with the same semantic function (with the code 120) parameterized by the concrete profile of `read_builtinIO`. Thus, the sort of a `read` term is known in the semantic procedure due to the profile of actually applied symbol `read_builtinIO`. The difference between two kinds of *built-in function* symbols is expressed by the sign of code numbers.

5.2.1 Booleans

At the beginning there is nothing, so ELAN provides the true and false values and introduces the bool module. These two values are built-in and are deeply connected to the implementation of conditions in rewrite rules.

`/elanlib/common/bool.eln`

Builtins	Code	Signature	Comments
<code>false</code>	0	<code>() bool</code>	The logical false .
<code>true</code>	1	<code>() bool</code>	The logical true .
<code>@ and @</code>	21	<code>(bool bool) bool</code>	The logical conjunction.
<code>@ or @</code>	22	<code>(bool bool) bool</code>	The logical disjunction.
<code>not(@)</code>	24	<code>(bool) bool</code>	The logical negation.
<code>@ == @</code>	8	<code>(bool bool) bool</code>	The logical equality test.
<code>@ != @</code>	9	<code>(bool bool) bool</code>	The logical inequality test.
<code>@ < @</code>	10	<code>(bool bool) bool</code>	A logical test assuming that <code>false < true</code> .
<code>@ <= @</code>	11	<code>(bool bool) bool</code>	A logical test assuming that <code>false < true</code> .
<code>@ > @</code>	12	<code>(bool bool) bool</code>	A logical test assuming that <code>false < true</code> .
<code>@ >= @</code>	13	<code>(bool bool) bool</code>	A logical test assuming that <code>false < true</code> .

`/elanlib/common/cmp.eln`

To enrich the booleans, *polymorphic* equality, disequality and inequalities are defined and are also built-in:

Builtins	Code	Signature	Comments
<code>@ == @</code>	18	<code>(X X) bool</code>	Tests the equality between two elements of the same sort X .
<code>@ != @</code>	19	<code>(X X) bool</code>	Tests the inequality between two elements of the same sort X .

5.2.2 Numbers

Numbers can of course be created “by hand”, but we choose in ELAN to provide built-in integers.

/elanlib/common/builtinInt.eln

Builtins	Code	Signature	Comments
@ + @	3	(builtinInt builtinInt) builtinInt	The addition on integers.
@ - @	4	(builtinInt builtinInt) builtinInt	The soustraction on integers.
@ * @	5	(builtinInt builtinInt) builtinInt	The multiplication on integers.
@ / @	6	(builtinInt builtinInt) builtinInt	The division on integers.
@ % @	27	(builtinInt builtinInt) builtinInt	The modulo operation.
@ & @	28	(builtinInt builtinInt) builtinInt	a & b makes the logical conjunction between the binary representations of a and b and returns the corresponding integer.
@ @	29	(builtinInt builtinInt) builtinInt	a b makes the logical disjonction between the binary representations of a and b and returns the corresponding integer.
- @	20	(builtinInt) builtinInt	The opposite of any integer.
@ == @	8	(builtinInt builtinInt) bool	The equality test.
@ != @	9	(builtinInt builtinInt) bool	The inequality test.
@ < @	10	(builtinInt builtinInt) bool	The strict ordering test between integers.
@ <= @	11	(builtinInt builtinInt) bool	The ordering test between integers.
@ > @	12	(builtinInt builtinInt) bool	The strict ordering test between integers.
@ >= @	13	(builtinInt builtinInt) bool	The ordering test between integers.
eq_builtin Int(@,@)	8	(builtinInt builtinInt) bool	The equality test.
neq_builtin Int(@,@)	9	(builtinInt builtinInt) bool	The inequality test.
less_builtin Int(@,@)	10	(builtinInt builtinInt) bool	The strict ordering test between integers.
lesseq_builtin Int(@,@)	11	(builtinInt builtinInt) bool	The ordering test between integers.

Builtins	Code	Signature	Comments
<code>greater_builtin</code> <code>Int(@, @)</code>	12	<code>(builtinInt builtinInt) bool</code>	The ordering test between integers.
<code>greaterreq_builtin</code> <code>Int(@, @)</code>	13	<code>(builtinInt builtinInt) bool</code>	The ordering test between integers.
<code>btoi(@)</code>	25	<code>(bool) builtinInt</code>	The conversion from a boolean to an integer (<i>false</i> becomes 0 and <i>true</i> becomes 1).
<code>itob(@)</code>	26	<code>(builtinInt) bool</code>	The conversion from any integer to a boolean (0 becomes <i>false</i> and any other <i>true</i>).

5.2.3 Identifiers

`/elanlib/noquote/ident.eln`

Builtins	Code	Signature	Comments
<code>@ == @</code>	14	<code>(ident ident) bool</code>	Tests the equality between two identifiers.
<code>@ != @</code>	15	<code>(ident ident) bool</code>	Tests the inequality between two identifiers.

`/elanlib/common/builtinString.eln`

Builtins	Code	Signature	Comments
<code>strlen(@)</code>	150	<code>(string) builtinInt</code>	Computes the length of a string.
<code>strcat(@, @)</code>	151	<code>(string string) string</code>	Concatenates two strings.
<code>@[@]</code>	152	<code>(string builtinInt)</code>	<code>s[i]</code> selects the <i>i</i> th character of the string <i>s</i> .
<code>@[@<-@]</code>	153	<code>(string builtinInt builtinInt) string</code>	<code>s[i<-c]</code> replaces the <i>i</i> th character of the string <i>s</i> by the character <i>c</i> .
<code>substr(@, @, @)</code>	154	<code>(string builtinInt builtinInt) string</code>	<code>substr(s, i, n)</code> extracts from the string <i>s</i> a substring of length <i>n</i> from the <i>i</i> th position.
<code>strspn(@, @)</code>	156	<code>(string string) builtinInt</code>	<code>strspn(s1, s2)</code> returns the length of the first part of <i>s1</i> entirely constituted by characters of <i>s2</i> .

Builtins	Code	Signature	Comments
<code>strcmp(@,@)</code>	157	<code>(string string) builtinInt</code>	<code>strcmp(s1,s2)</code> compares two strings character by character and returns an integer (a negative one if <code>s1</code> is less than <code>s2</code> , 0 if they are equal and otherwise a positive integer).
<code>string(@)</code>	158	<code>(builtinInt) string</code>	Conversion from an integer (that is, the associated character) to a string.
<code>ident2string(@)</code>	177	<code>(ident) string</code>	Conversion from an identifier to a string.

5.2.4 Elementary term computations

Since it is of primarily use in symbolic computation on terms (and remember that everything in ELAN is a term except the built-ins), the occurrence relation and the replacement operation are provided as built-ins.

/elanlib/common/occur.eln

The module `occur[X,Y]` is parameterized by two sorts `X,Y` that must be non built-in.

Builtins	Code	Signature	Comments
<code>occurs @ in @</code>	17	<code>(X Y) bool</code>	<code>occurs s in t</code> checks if <code>s</code> occurs in <code>t</code> .

/elanlib/common/replace.eln

The module `replace[X,Y]` is parameterized by two sorts `X,Y` that must be non built-in.

Builtins	Code	Signature	Comments
<code>replace @ by @ in @</code>	16	<code>(X X Y) Y</code>	<code>replace s by u in t</code> replaces all occurrences of <code>s</code> by the <code>u</code> in <code>t</code> .

/elanlib/common/builtinSyntacticMatching.eln

Matching and unification operations are provided as built-ins.

Builtins	Code	Signature	Comments
<code>builtinSyntacticMatching(@,@,@,@)</code>	191	<code>(X X Y Y) Y</code>	<code>builtinSyntacticMatching(t1,t2,vars,failure)</code> tries to match terms <code>t1</code> to <code>t2</code> (i.e. <code>t1 << t2</code>), where <code>vars:Y</code> represents list of variables occurred in <code>t1:X</code> and <code>t2:X</code> and <code>failure:Y</code> is a value returned in a non-successful case. Otherwise, a matching substitution is returned in the form of a list of terms associated to the list of variables <code>vars</code> . The sort <code>Y</code> is supposed to be <code>list[X]</code> . It works only for empty theories.

Builtins	Code	Signature	Comments
<code>builtinSyntacticUnification(@,@,@,@)</code>	175	$(X\ X\ Y\ Y)\ Y$	the same as the previous built-in function, except that a most general unifier is searched.

5.2.5 Input/Output

`/elanlib/common/Query.eln`

The module `Query[I,O,P]` defines the parameterized sort `Query[I,O,P]` and the sort `ide` (embedding the sort `ident` of identifiers). All symbols defined in this module are built-in constructors w.r.t. the classification mentioned before.

Builtins	Code	Signature	Comments
<code>quit</code>	90	$()\ \text{Query}[I,O,P]$	<code>quit</code> just quit the command interpreter
<code>help</code>	112	$()\ \text{Query}[I,O,P]$	<code>help</code> prints the help menu (in the file <code>elanlib/help.txt</code>)
<code>load @</code>	91	$(\text{ide})\ \text{Query}[I,O,P]$	<code>load f</code> loads the file <code>f.eln</code> and extends the current specification
<code>batch @</code>	107	$(\text{ide})\ \text{Query}[I,O,P]$	<code>batch f</code> runs commands of the batch file <code>f.eln</code>
<code>run @</code>	94	$(I)\ \text{Query}[I,O,P]$	<code>run q</code> runs the current specification with the query <code>q</code>
<code>sorts @ @ @</code>	92	$(\text{ide}\ \text{ide}\ \text{ide})\ [I,O,P]$	<code>sorts q r s</code> redefines sorts of query, results and print-outs
<code>startswith (@)@</code>	93	$(\text{ident}\ O)\ \text{Query}[I,O,P]$	<code>startswith (s)t</code> redefines the starting term $(s)t$ of the <code>.lgi</code> file, where <code>s</code> is a strategy and <code>t</code> is a pattern of the input query
<code>checkwith @</code>	102	$(\text{bool})\ \text{Query}[I,O,P]$	<code>checkwith b</code> redefines the checking term of the <code>.lgi</code> file, where <code>b</code> is a pattern of a boolean condition tested before any input query
<code>printwith @</code>	98	$(P)\ \text{Query}[I,O,P]$	<code>printwith t</code> redefines the printing term used to pretty-print any result
<code>qs</code>	100	$\text{Query}[I,O,P]$	dump the stack of queries
<code>rs</code>	101	$\text{Query}[I,O,P]$	dump the stack of results
<code>stat</code>	96	$\text{Query}[I,O,P]$	print statistics
<code>dump</code>	95	$\text{Query}[I,O,P]$	dump all rules
<code>dump @</code>	108	$(\text{ident})\ \text{Query}[I,O,P]$	<code>dump(l)</code> dumps rules or strategies with the label/name <code>l</code>
<code>dump @</code>	109	$(\text{builtinInt})\ \text{Query}[I,O,P]$	<code>dump(s)</code> gives all information about the symbol <code>s</code> given by its code
<code>display @</code>	103	$(\text{builtinInt})\ \text{Query}[I,O,P]$	<code>display m</code> changes the printing mode of symbols: <code>display 1</code> shows terms in internal form, while <code>display 0</code> uses the traditional representation

Builtins	Code	Signature	Comments
<code>trace @</code>	97	<code>(builtinInt) Query[I,0,P]</code>	<code>trace n</code> changes the level of debugging to level <code>n</code>
<code>break @</code>	104	<code>(ident) Query[I,0,P]</code>	<code>break f</code> sets a break-point on rules or strategies with the label/name <code>f</code>
<code>break @</code>	110	<code>(builtinInt) Query[I,0,P]</code>	<code>break s</code> sets a break-point on the symbol <code>s</code> given by its code
<code>unbreak @</code>	105	<code>(ident) Query[I,0,P]</code>	<code>unbreak f</code> deletes the break-point on rules or strategies with the label/name <code>f</code>
<code>unbreak @</code>	111	<code>(builtinInt) Query[I,0,P]</code>	<code>unbreak s</code> deletes the break-point on the symbol <code>s</code>
<code>breaks</code>	106	<code>() Query[I,0,P]</code>	lists all set break points

/elanlib/common/IO.eln

The module `IO[X]` defines the Input/Output primitives which can only be used with the compiler. This module is parameterized by the sort `X` of the terms used in input/output operations. Currently, only the following sorts are available in the input/output operations: `string`, `int`, `bool`, `term`. For example, to write out or read in strings, the user only need to import `IO[string]` in his or her program.

Builtins	Code	Signature	Comments
<code>write(@,@)</code>	-119	<code>(File X) X</code>	Writes on output <code>File</code> the argument of sort <code>X</code> .
<code>writeln(@,@)</code>	-119	<code>(File X) X</code>	Writes on output <code>File</code> the argument of sort <code>X</code> and goes to the next line.
<code>read(@)</code>	-120	<code>(File) X</code>	Reads from input <code>File</code> and return an element of sort <code>X</code> .
<code>Error(@)</code>	123	<code>(builtinInt) X</code>	Reports various errors of the I/O sub-system written in C++.

/elanlib/common/builtinStdio.eln

The module `builtinStdio.eln` provides several input/output primitives. But except for `open` and `close`, the others can and should be replaced by the primitives in `IO`. Notice that the module `builtinStdio.eln` is already imported by `IO` and do not need to be explicitly imported in ELAN programs. Three constants of sort `File` `stdin`, `stdout` and `stderr` provide respectively the standard input, the standard output and the standard error output.

Builtins	Code	Signature	Comments
<code>getc(@)</code>	113	<code>(builtinInt) builtinInt</code>	Gets the next character from an input file or pipe.
<code>putc(@)</code>	114	<code>(builtinInt builtinInt) builtinInt</code>	Puts a character to an output file or pipe.
<code>create(@)</code>	115	<code>(builtinString) builtinInt</code>	Creates a process with a given name. The created process is linked with the parent process via two blocking pipes.
<code>create_noblock(@)</code>	117	<code>(builtinString) builtinInt</code>	Creates a process with a given name such that two communication pipes are in a non blocking mode.
<code>open(@,@)</code>	116	<code>(builtinString builtinString) File</code>	Opens a file for reading or writing. The first argument is the name of the file, and the second argument is either "w" (write or create mode), "r" (read mode) or "a" (append mode).
<code>close(@)</code>	118	<code>(File) builtinInt</code>	Closes a file or kill a process denoted by the pipe number of sort File.

5.2.6 Strategies

`/elanlib/strategy/Meta_strat.eln`

The module `Meta_strat[X]` parameterized by a sort `X` can only be used in interpreted mode. This module exports the sort `Strategy[X]`, which specifies an external form of basic strategies of sort `X`. The signature of basic strategies is defined by several built-in constructors used to convert strategy terms of sort `Strategy[X]` into basic strategies over the sort `X`. This conversion is used for the implementation of `meta_apply` symbols of the previous module.

Builtins	Code	Signature	Comments
@	131	(Strategy[X]) Strategies[X]	Embeds a basic strategy in a list of basic strategies.
@ , @	132	(Strategy[X] Strategies[X]) Strategies[X]	Concatenates a basic strategy to a list of basic strategies.
@	133	(string) Labels[X]	Embeds a string in a list of rule labels.
@ , @	134	(string Labels[X]) Labels[X]	Concatenates a string to a list of rule labels.
dc(@)	135	(Labels[X]) Strateg[X]	dc(1) is a basic strategy that chooses one label in the list of labels 1 and returns all results of the corresponding rule.
dk(@)	136	(Labels[X]) Strateg[X]	dk(1) is a basic strategy that chooses successively all labels in the list of labels 1 and returns all results of the corresponding rules.
dc(@)	137	(Strategies[X]) Strateg[X]	dc(1s) is a basic strategy that chooses one basic strategy in the list 1s and returns all its results.
dk(@)	138	(Strategies[X]) Strateg[X]	dk(1s) is a basic strategy that chooses successively all basic strategies in the list 1s and returns all their results.
repeat*(@)	139	(Strategy[X]) Strateg[X]	repeat*(s) is a basic strategy that repeats the application of the basic strategy s until it fails.
iterate*(@)	140	(Strategy[X]) Strateg[X]	iterate*(s) is a basic strategy that repeats the application of the basic strategy s until it fails and returns intermediate results.
@	141	(Strateg[X]) Strategy[X]	Embeds the previous constructions into the sort Strategy[X].
@ ; @	142	(Strateg[X] Strategy[X]) Strategy[X]	Concatenates any previous construction to a strategy.
id	143	() Strateg[X]	id is a basic strategy.
call @ : X	144	(string) Strateg[X]	call s:X represents an application of a basic strategy named s of sort X.

/elanlib/strategy/Meta_apply.eln

The module `Meta_apply[X]` parameterized by a sort `X` can only be used in interpreted mode (cf. the module `Meta_capply[X]` for the compiled mode).

The construction `where y:=(META)meta_apply(s,t)` exported from this module is semantically equivalent to `where y:=(s)t` where `s` is a basic strategy, i.e. it gives all results of application `s` on `t`. There exists also a slightly modified construction `where y:=(META)meta_apply(s,t,n)` giving only the `n`-th solution of the application of `(s)t`, if it exists.

The ELAN built-in strategy `META` used here just indicates that `(META)meta_apply(s,t)` returns in general several results, which would not be the case with `()meta_apply(s,t)`. The built-

in strategy META is applicable only to terms of the form `meta_apply(s,t)` or `meta_apply(s,t,n)`.

The module `Meta_apply.eln` exports several variants of the symbol `set_of(s,t,m,n)` returning a list of `m` results of the application `(s)t` from the `n`-th result. The symbols `set_of(s,t)`, `set_of(s,t,m)` and `set_of(s,t,m,n)` are implemented using a result-collecting built-in symbol `meta_apply(s,ts,m,n):(Strategy[X] list[X] builtinInt builtinInt) list[X]` defined in this module.

Builtins	Code	Signature	Comments
<code>meta_apply(@,@)</code>	-129	<code>(Strategy[X] X) X</code>	<code>meta_apply(s,t)</code> returns all results of application of the basic strategy <code>s</code> on the term <code>t</code> .
<code>meta_apply(@,@,@)</code>	-130	<code>(Strategy[X] X builtinInt) X</code>	<code>meta_apply(s,t,n)</code> returns the <code>n</code> -th result of application of the basic strategy <code>s</code> on the term <code>t</code> , if it exists.
<code>meta_apply(@,@,@,@)</code>	-128	<code>(Strategy[X] list[X] builtinInt builtinInt) list[X]</code>	The local built-in symbol <code>meta_apply(s,t.nil,m,n)</code> returns <code>n</code> results starting from the <code>m</code> -th of application of the basic strategy <code>s</code> on the term <code>t</code> , if they exist.

/elanlib/strategy/Meta_capply.eln

The module `Meta_capply[X]` parameterized by a sort `X` can only be used in compiled mode. It represents a simplified form of the module `Meta_apply[X]`, where a construction of a strategy of sort `Strategy[X]` is restricted to a reference of an existing strategy (i.e. a strategy already occurring in the compiled program). Thus, in a `meta_apply` call, only a name (i.e. a string) of a referenced strategy is used instead of a strategy term of sort `Strategy[X]`.

Builtins	Code	Signature	Comments
<code>meta_apply(@,@)</code>	-129	<code>(string X) X</code>	<code>meta_apply(s,t)</code> returns all results of application of the strategy named <code>s</code> on the term <code>t</code> .
<code>meta_apply(@,@,@)</code>	-130	<code>(string X builtinInt) X</code>	<code>meta_apply(s,t,n)</code> returns the <code>n</code> -th result of application of the strategy named <code>s</code> on the term <code>t</code> , if it exists.

/elanlib/strategy/strsig.eln

The module `strsig[X,Y]` parameterized by the two sorts `X,Y` defines the signature of the defined strategies. It introduces several built-in constructors used to construct strategy terms of the strategy language. The module `strsig.eln` contains strategy constructors used both for type-preserving and type-changing strategies.

Builtins	Code	Signature	Comments
[@]@	-180	($\langle X \rightarrow Y \rangle$ X) Y	[S]t applies the strategy S to the term t.
dk(@)	-182	(Strlist[X,Y]) $\langle X \rightarrow Y \rangle$	dk(1) is the defined strategy that chooses successively all strategies in the list 1 and returns all their results.
dc(@)	-181	(Strlist[X,Y]) $\langle X \rightarrow Y \rangle$	dc(1) is the defined strategy that chooses one strategy in the list 1 and returns all its results.
dc one(@)	-190	(Strlist[X,Y]) $\langle X \rightarrow Y \rangle$	dc one (1) is the defined strategy that chooses one strategy in the list 1 and returns one result.
fail	-183	() $\langle X \rightarrow Y \rangle$	The defined strategy that always gives an empty set of results.
@,@	-185	($\langle X \rightarrow Y \rangle$ Strlist[X,Y]) Strlist[X,Y]	Concatenates a defined strategy to a list of strategies.
Epsilon	-189	() Strlist[X,Y]	The empty list of strategies.
if @ then @ else @ fi	-184	(bool $\langle X \rightarrow Y \rangle$ $\langle X \rightarrow Y \rangle$) $\langle X \rightarrow Y \rangle$	if b then s1 else s2 fi is the defined strategy that chooses either s1 if b is true otherwise s2.
@	146	(Assignment) AssignmentList	Embeds an assignment in a list of assignments.
@ , @	147	(AssignmentList Assignment) AssignmentList	Add an assignment to the end of a list of assignments.
if @	145	(bool) IfApplication	A condition if an IfApplication.
@	146	(IfApplication) AssignmentList	Embeds an IfApplication in a list of assignments.
@ , @	147	(AssignmentList IfApplication) AssignmentList	Add an IfApplication to the end of a list of assignments.

/elanlib/strategy/strconc.eln

The module `strconc[X,Y,Z]` parameterized by three sorts X,Y,Z introduces a concatenation symbol ; of two strategies of sorts $\langle X \rightarrow Y \rangle$ and $\langle Y \rightarrow Z \rangle$.

Builtins	Code	Signature	Comments
@ ; @	-188	($\langle X \rightarrow Y \rangle$ $\langle Y \rightarrow Z \rangle$) $\langle X \rightarrow Z \rangle$	s1 ; s2 concatenates the strategies s1: $\langle X \rightarrow Y \rangle$ and s2: $\langle Y \rightarrow Z \rangle$ to get a strategy of sort $\langle X \rightarrow Z \rangle$.

/elanlib/strategy/strat.eln

The module `strat[X]` parameterized by a sort X has to be imported for working with type-preserving strategies of sort $\langle X \rightarrow X \rangle$. It defines the `eval` strategy of the interpreter of the language of defined strategies, and several specific strategy constructors, which are not present in the case of type-changing strategies. The rest of strategy constructors is defined in the module `strategy/strsig.eln`.

Builtins	Code	Signature	Comments
id	-186	() <X->X>	Identity strategy.
if @ then @ orelse @ fi	-187	(<X->X> <X->X> <X->X>) <X->X>	Orelse strategy.

5.2.7 REF Format

/elanlib/ref/Meta_Apply.eln

The module `ref/Meta_Apply.eln` provides several variants of the function `meta_apply`, which apply a strategy on a term in an environment defined by a program. These three components are either encoded in the REF-format or passed as strings. Results are produced equally in the REF-format, or as strings representing terms.

Builtins	Code	Signature	Comments
<code>meta_apply(@,@,@,@,@)</code>	170	(string string string string builtinInt) string	<code>meta_apply(s,t,file,spec,n)</code> gives the <i>n</i> -th result of the application of the strategy <i>s</i> on the term <i>t</i> w.r.t. the program stored in files named <i>file.lgi</i> and <i>spec.spc</i> .
<code>meta_apply(@,@,@,@,@)</code>	171	(string list[string] string string builtinInt) string	<code>meta_apply(s,t.nil,file,spec,n)</code> gives the <i>n</i> -th result of the application of the strategy <i>s</i> on the term <i>t</i> w.r.t. the program in files named <i>file.lgi</i> and <i>spec.spc</i> .
<code>meta_apply(@,@,@,@)</code>	172	(string string string builtinInt) string	<code>meta_apply(s,t,file,n)</code> gives the <i>n</i> -th result of the application of the strategy <i>s</i> on the term <i>t</i> w.r.t. the program in the REF-format stored in file named <i>file.ref</i> .
<code>meta_apply(@,@,@,@,@)</code>	173	(string list[string] string string builtinInt) string	<code>meta_apply(s,t.nil,file,n)</code> gives the <i>n</i> -th result of the application of the strategy <i>s</i> on the term <i>t</i> w.r.t. the program in the REF-format stored in file named <i>file.ref</i> .

/elanlib/ref/ref2string.eln

The module `ref/ref2string.eln` provides two conversion functions between strings of REF-terms and terms of the user's defined signature. It also exports a pretty-printing and a parsing function parameterized by a signature in the REF-format.

Builtins	Code	Signature	Comments
<code>refstring2term(@)</code>	-168	<code>(string) X</code>	<code>refstring2term(s)</code> builds a term of sort <code>X</code> from the string <code>s</code> .
<code>term2refstring(@)</code>	-167	<code>(X) string</code>	<code>term2refstring(t)</code> transforms the term <code>t</code> of sort <code>X</code> into a string.
<code>term2string(@)</code>	-169	<code>(X) string</code>	<code>term2string(t)</code> pretty-prints the term <code>t</code> of sort <code>X</code> .

/elanlib/ref/refstring2string.eln

The module `ref/refstring2string.eln` provides built-in pretty-printing and parsing functions converting strings of terms into strings of REF-terms, and vice-versa. This module defines also two symbols `spec2ref` and `ThisProgram` constructing a REF-representation of a program.

Builtins	Code	Signature	Comments
<code>spec2ref(@,@)</code>	162	<code>(string string) string</code>	<code>spec2ref(file1,file2)</code> builds a REF-file from the specification described by two file <code>file1.spc</code> and <code>file2.lgi</code> , and it returns its name <code>file.ref</code> .
<code>ThisProgram</code>	165	<code>() Program</code>	Produces the REF-format of the running program.
<code>refstring2string(@,@)</code>	160	<code>(string string) string</code>	<code>refstring2string(s,file.ref)</code> pretty-prints <code>s</code> according to the grammar defined in the file <code>file.ref</code> .
<code>string2refstring(@,@,@)</code>	161	<code>(string string string) string</code>	<code>string2refstring(st:so,file.ref)</code> parses the string <code>st</code> of a term of sort <code>so</code> according to the grammar defined in the file <code>file.ref</code> .

5.3 Calling external programs

ELAN allows using external programs like for instance the UNIF system [Rin97]. Let us describe how it works.

The external program is assumed to take as input a term and to return to ELAN a set of terms. One could imagine the input term as an input constraint and the returned terms as the solved forms computed by the program. The program is called as a sub-process of the ELAN session and it is linked to it via two UNIX pipes which are respectively attached to the standard input and output of the process (i.e. `stdin`, `stdout`).

Since it is useful to reuse a given process as much as possible, the description of the called process gives the number of terms that can be sent to the process before killing it and replacing it by another one.

Data (i.e. terms) are sent in and out under a textual format. This format is determined by the signature of the module in which the process call is performed.

The process call is realized through the use of the strategies `dccall` and `dkcall` (that stand for *dont care call* and *dont know call*), with three arguments:

- the first is the name of the process,

- the second is the number of terms that should be transmitted to the process before killing it,
- and the third is the (unique) sort of initial and result terms.

The syntax is therefore the following:

$$\begin{aligned} \langle \text{strategy} \rangle &::= \text{dkcall}(\langle \text{identifier} \rangle, \langle \text{number} \rangle, \langle \text{sort name} \rangle) \\ &\quad | \quad \text{dccall}(\langle \text{identifier} \rangle, \langle \text{number} \rangle, \langle \text{sort name} \rangle) \end{aligned}$$

The difference between the two calls is that the first one returns all the computed results while the second one returns only the first result, even if several are computed.

In order to be able to communicate with ELAN, the process which is called should obey to the following syntactic restrictions.

The output should have the following syntax:

$$\begin{aligned} \langle \text{process output} \rangle &::= \{ \# \}^+ \langle \text{next solution} \rangle \\ \langle \text{next solution} \rangle &::= \langle \text{number} \rangle \{ * \}^+ \langle \text{term} \rangle \{ \# \}^+ \langle \text{next solution} \rangle \\ &\quad | \quad \text{END} \{ \# \}^+ \end{aligned}$$

where $\langle \text{term} \rangle$ is one result.

In the same way, the process should accept the syntax of the initial term, described as follows:

$$\begin{aligned} \langle \text{process input} \rangle &::= \langle \text{term} \rangle \langle \text{next solution demand} \rangle \\ \langle \text{next solution demand} \rangle &::= . \langle \text{next solution demand} \rangle \\ &\quad | \quad ; \end{aligned}$$

where $\langle \text{term} \rangle$ is the term passed to the built-in process.

The way the communication is organised between ELAN and the built-in process is as follows. ELAN first sends the initial term to the process and every time it needs another result, it sends the symbol “.” and waits for the answer of the process on its standard output. The answer should be in the syntax corresponding to the non-terminal $\langle \text{next solution} \rangle$. If ELAN does not need anymore solution, it sends the symbol “;” which makes the process stop, or it sends an “end of file” if the limit of the process use is reached.

▷ Since the synchronization between ELAN and the called process is made through pipes, the two process should reinitialise their buffers after each large output. As a consequence the called process should call the `fflush(stdout)` procedure after returning each solution.

Example 5.1 Assume that one would like to use the shell command “sort” for sorting a list of “elements”. The first step is to create a file containing the shell script calling the sort command and that follows the syntactic conventions required by ELAN as just described above. Let us call this file *elansort*:

```
#!/bin/sh
cat /dev/null > /tmp/inputFile
read r
while (test "$r" != "") do
  echo $r >> /tmp/inputFile
  read r
done
echo "##0**"
```

```

sort < /tmp/inputFile
echo "##END#"
read r
while (test "$r" != ";") do
  read r
done
/bin/rm /tmp/inputFile

```

Then, we construct the syntax of “elements” by using a preprocessor instruction involving a list (*Vars*) of identifiers taken from a specification file:

```

import  global  List ;
        local   Vars anyIdentifier list[identifier] ;
end

operators global
  FOR EACH SS:identifier SUCH THAT SS:=(listExtract) elem(Vars) :{
    SS      : elem ;
  }
end

```

Lists are defined as sequences of elements, without any separator:

```

sort list; end

operators global
  @          : ( elem ) list;
  @ @        : ( list list ) list  assocLeft;
end

```

Since *elansort* could only be used for sorting elements given successively on each line of a file, we have to convert lists in the right syntax before calling the external program:

```

module listSort[Vars]
import      global      Elem[Vars] ;
        local      List callSort[Vars] ;
end

operators local
  @ '\10'      : ( list ) list;
  @ newline    : ( list ) list alias @ '\10': ;
  convert(@)   : ( list ) list ;
end

stratop global
strataddnl : < list -> list > bs ;
extSort: < list -> list > bs ;
end

rules for list
  P,Q : list ;
  a: elem ;

global
[addnewline] P => P newline end
[]          convert(P a) => convert(P) Q
          where Q:=() a newline end

```

```

[]          convert(a) => a newline end
[conversion] P => Q where Q := (strataddnl) convert(P) end
[sortRule]  P => Q where Q := (callSort) P end
end

strategies for list
implicit
[] strataddnl => dc one(addnewline) end
[] extSort => dc one(conversion) ; dc one(sortRule) end
end
end

```

The main strategy `extSort` applies first the conversion rule and then applies the rule calling the external program through the `callSort` strategy defined in the following ELAN module:

```

module callSort[Vars]
import      global      Elem[Vars] ;
      local      Vars anyIdentifier list[identifier]
      List ;
end

operators local
  FOR EACH SS:identifier SUCH THAT SS:=(listExtract) elem(Vars) :{
    SS      : elem              alias SS: ;
  }
  @
  @ @      : (elem ) list alias @:;
  @ @      : (list list) list assocLeft alias @ @:;
end

stratop global
callSort : < list -> list > bs ;
end

strategies for list
implicit
[] callSort => dkcall(elansort,1,list) end
end
end

```

This module can be considered as an encapsulation of the `elansort` program. According to the alias declarations, results are written in the initial syntax, as sequences of elements separated by spaces.

Chapter 6

Contributed works

6.1 Description of the **ELAN** contributions

Many computational processes in Automated Deduction and Constraint Programming can be expressed as instances of a general approach that consists of applying transformation rules on formulas with some strategy, until reaching specific normal forms. Such processes are naturally modelled in ELAN, and can be classified according to the area of interest, namely programming, proving and solving.

Programming: One of the first application was to prototype the fundamental mechanisms of logic and functional programming languages like first-order resolution and λ -calculus. The general framework of Constraint Logic Programming can be easily designed in the ELAN framework [KR98], since its operational semantics is clearly formalised as rewrite rules, although the application strategy is often defined in an informal way. Some implementations [Bor95] related to a calculus of explicit substitutions (the first-order rewrite system $\lambda\sigma$ that mimics λ -calculus) open the way of implementing higher-order logic programming languages via a first-order setting. Another calculus of explicit substitutions based on the π -calculus is used to provide a formal specification of Input/Output for ELAN [Vir96].

Proving: ELAN was used in order to implement a predicate prover based on the rules proposed by J.-R. Abrial, and implemented in the B-tools [CK97]. We developed also a propositional sequent calculus, completion procedures for rewrite systems [KM95], sufficient conditions for the termination problem [GG97]. A library for automata construction and manipulation has been designed. Approximation automata are used to check conditions for reachability, sufficient completeness, absence of conflicts in systems described by non-conditional rewrite rules [Gen98b, Gen98a].

Checking: A very special case of proof is just exhaustive check of all the possibilities. Using in particular “dont know” strategies, it was efficiently rediscovered by exhaustive search that the well known Needham and Schroeder authentication protocol is unsafe [Cir99].

Solving: The notion of rewriting controlled by strategies is used in [Cas98a, KR98] to describe in a unified way the constraint solving mechanism as well as the meta-language needed to manipulate the constraints [Cas97a, CK98]. This provides programs that are very close to the proof theoretical setting used now to describe constraint manipulations like unification or numerical constraint solving. ELAN offers a constraint programming environment where the formal description of a constraint solver is directly executable. ELAN has been tested on several examples of constraint solvers for various computation domains and combinations like abstract domains [KR98, Rin97] (term algebras) and more concrete ones (finite domains, integers). In [Cas96a, Cas96b, Cas97b, Cas98c], it is shown how to use computational systems as a general framework for handling Constraint Satisfaction Problems (CSP for short). The approach

leads to the design in ELAN of COLETTE, a solver for constraints over integers and finite domains [Cas98b]. We also combine rules, constraints and strategies in order to deal with problems like planning and scheduling [DK99].

6.2 ELAN's annotated bibliography

- [BKK⁺96b, BKK97, BKK⁺98b] General presentations of the different ELAN system releases.
- [BKK96a, BK97, Bor98b] A description of the operational as well as denotational semantics of the new ELAN strategies, available since version 2.0. This allows the user to define its own strategies as a computational system.
- [BKK98a] Towards a functional operational semantics of ELAN.
- [BJMR98] Some useful applications related to the REF format.
- [BC98] The facilities provided in ELAN for the cooperation of constraint solvers.
- [Bor98a] A full description of the ELAN strategy language. In french.
- [Bor95] An ELAN implementation of higher-order unification by using a calculus of explicit substitutions.
- [Cas96b, Cas96a, Cas97b] The use of ELAN to specify various constraint manipulation algorithms for CSP's.
- [Cas98a, KR98, Cas97b, Cas98b, Cas98c] The rule-based approach to implement Constraint Programming and Constraint Solving Techniques.
- [CK99a, CK99b] The presentation of the rewriting calculus that provides a full semantics to ELAN.
- [CK97] The specification of Abrial's B predicate prover.
- [DK98] The strategy language is used to build plans.
- [DK99] A combination of rewriting rules, constraints and strategy language for planning problems.
- [KKV95] The first presentation of the general ideas developed in ELAN, with the definition of computational systems (including the definition of strategies) and the application of ELAN to design and to prototype constraint programming languages.
- [KM96] An illustration of the reflective power of ELAN and rewriting logic.
- [KM95] The ELAN implementation of some rule-based completion procedures.
- [Kir97] An ELAN tutorial.
- [MK97, MK98, KM98, Mor99] A description of the new compilation techniques developed for ELAN, in particular for associativity and commutativity
- [Rin97] The use of ELAN to interface various unification algorithms and to build combination algorithms for unification.

-
- [Vir96] An implementation of input/output in ELAN via an explicit substitution calculus for the π -calculus.
- [Vit96] A description of the first ELAN compiler.
- [Vit94] The first main version of the ELAN system in full details, from design to implementation. In French.

Acknowledgements

We are very grateful to Mark van den Brand whose comments, enlighting questions and always pertinent suggestions allowed us to come up with a much better state of this manual. Many thanks to Carlos Castro, Thomas Genet and Christelle Scharff for many fruitful discussions and constructive criticisms about ELAN, for their careful reading of the manual and checking of examples. Many thanks also to Jounaidi Benhassen for his feedback. Any remaining errors are of course entirely ours.

Specials thanks are going to José Meseguer for the many discussions we had on rewriting logic and rewrite based programming languages and to Steven Eker for his participation to a early stage of this project and for always useful interactions.

Many thanks also to all ELAN's users for their feedback on the language and the system.

Index

- `:=`, 37
- AC, 34
- AC**, 34, 36
- alias
 - strategy, 35
- alias**, 32–35
- aliasing, 32, 35
- AND**, 60
- ANDIF**, 60
- <arg.repetition>*, 59
- assocLeft**, 34
- assocLeft**, 34, 36
- assocRight**, 34
- assocRight**, 34, 36
- <boolean term>*, 37, 57, 60
- bs**, 36, 37, 41
- builtin**, 34, 35
- <char>*, 29, 30, 59
- check with**, 57
- choose**, 37
- choose-try, 39
- <choosing>*, 41
- code, 34
- code**, 34, 35
- Coercions, 34
- comments, 30
- <concatenation>*, 41
- condition, 36
- <constant expression>*, 59
- dc**, 41
- dc one**, 41, 42
- dccall**, 82, 83
- declaration
 - function, 32
 - operator, 32
 - sort, 31
 - strategy, 35
- defined
 - strategy, 47
- defined strategies, 40
- <defined strategy option>*, 36
- <defined strategy rule>*, 40, 47
- description**, 57
- dk**, 41, 52
- dkcall**, 82, 83
- <Dstrategy term>*, 37, 47
- <elan identifier>*, 29, 32, 37
- elementary strategies, 40
- elementary strategy, 41
- <elementary strategy option>*, 36
- <elementary strategy rule>*, 40, 41
- END**, 83
- end**, 30, 36, 37, 40, 41, 47, 57
- end-user, 7
- <Estrategy name>*, 41, 57
- <Estrategy term>*, 37, 41
- evaluation
 - mechanism in ELAN, 48
- explicit**, 40, 41
- fail
 - strategy, 40
- fail**, 41
- <family of rules>*, 30, 36
- <family of strategies>*, 30, 40
- first**, 41
- first one**, 41, 42
- FOR EACH**, 60
- <for each const.>*, 60
- <formal module name>*, 30, 56
- function symbol, 32
- global**, 36
- <global operator definition>*, 31
- <global stratop definition>*, 31
- id**, 41
- <identifier>*, 29–31, 33, 36, 56, 57, 59, 83
- if, 36

- if**, 37
- <if-where-choose>*, 37, 47
- implicit**, 40, 41
- import**, 57
- <imports>*, 30
- <instantiated module name>*, 30, 57
- iterate***, 41, 44
- iterate+**, 41, 44
- <iterator>*, 41
-
- <labelled or unlabelled rule>*, 36
- <labelled rule>*, 36
- labelled rules, 36
- letter, 29
- <letter>*, 29
- <lexem>*, 59
- lexical unity, 29
- <LGI file name>*, 27
- local**, 36
- local assignment, 36
- <local operator definition>*, 31
- <local stratop definition>*, 31
- LPL**, 57
- <lpl description>*, 57
-
- <module>*, 30
- module**, 30
-
- name
 - sort, 30, 31
- <name>*, 32, 35
- <nchar>*, 29, 32
- <new operator declaration>*, 32
- <new strategy declaration>*, 35
- <next solution>*, 83
- <next solution demand>*, 83
- normalise**, 41
- <normalize>*, 41
- normalize**, 41
- <normalize with trace>*, 41
- normin**, 25, 41, 46
- number, 30
- <number>*, 30, 32, 34, 36, 59, 83
- <numeral>*, 29, 30
-
- of sort**, 57
- <old name>*, 32
- <old operator name>*, 32
- <old strategy name>*, 35
- operator, 32
 - <operator alias>*, 32
 - <operator declaration>*, 32
 - operator definition, 31
 - <operator definition>*, 30
 - <operator name>*, 32
 - <operator option>*, 32, 34
 - <options>*, 27
-
- part**, 57
- pattern
 - in where, 38
- patterns, 38
- pri, 34
- pri**, 34, 36
- primal strategy, 42
- priority, 34
- <process input>*, 83
- <process output>*, 83
-
- <qchar>*, 29, 30
- query**, 57
- <query term>*, 57
- <quoted identifier>*, 29, 32
- <quoted single lexem>*, 32
-
- rank, 32
 - strategy, 35
- <rank>*, 32, 33
- <REF file name>*, 27
- repeat**, 44
- repeat***, 41, 44
- repeat+**, 41, 44
- result of sort**, 57
- <rule body>*, 36, 37
- <rule label>*, 36, 47
- rules
 - labelled, 36
 - unlabelled, 36
- rules for**, 36
-
- selectors, 32
- separators, 29
- <simple repetition>*, 59
- <single lexem>*, 32
- sort
 - built-in, 31
 - declaration, 31
 - definition, 30
 - in pattern, 38
 - name, 30, 31

- strategy, 35
- sort**, 30
- sort declaration, 31
- <sort definition>*, 30
- <sort name>*, 30, 31, 33, 35–37, 40, 41, 57, 60, 66, 83
- <sort or strategy sort name>*, 41
- <specification description>*, 57
- specification description**, 57
- <specification part description>*, 57
- <start term>*, 57
- start with**, 57
- strategies for**, 40
- strategy, 40
 - defined, 40, 47
 - elementary, 40, 41
 - failure, 40
 - family, 40
 - primal, 42
- <strategy>*, 83
- <strategy alias>*, 35
- <strategy arg>*, 35
- <strategy args>*, 35
- <strategy body>*, 47
- <strategy declaration>*, 35
- <strategy name>*, 35, 60, 66
- strategy operator, 35
- <strategy option>*, 35, 36
- <strategy rank>*, 35
- <strategy sort name>*, 35, 36, 41
- <strategy variable declare>*, 40, 41
- stratop definition, 31
- <stratop definition>*, 30
- <subexpression>*, 59
- SUCH THAT**, 60
- super-user, 7
- <symbol>*, 32
-
- <term>*, 37, 60, 83
- <text>*, 59, 60
- try**, 37
-
- <unlabelled rule>*, 36
- unlabelled rules, 36
- user, 7
-
- <varaffect>*, 60
- <vardecl>*, 60
- <variable>*, 36
- <variable declare>*, 36, 37
-
- <variable name>*, 37, 41
- <varname>*, 60
- visibility
 - of alias*, 55
 - of operator*, 55
 - of sort*, 55
 - rule labels, 40
-
- where, 38
 - with pattern, 38
- where**, 37

Bibliography

- [BC98] P. Borovanský and C. Castro. Cooperation of Constraint Solvers: Using the New Process Control Facilities of ELAN. In C. Kirchner and H. Kirchner, editors, *Proc. Second Intl. Workshop on Rewriting Logic and its Applications*, Electronic Notes in Theoretical Computer Science. Elsevier, September 1998.
- [BCD⁺98] P. Borovanský, H. Cirstea, H. Dubois, C. Kirchner, H. Kirchner, P.-E. Moreau, C. Ringeissen, and M. Vittek. *User Manual of the ELAN Standard Library*. LORIA, Nancy (France), first edition, December 1998.
- [BJMR98] P. Borovanský, S. Jamoussi, P.-E. Moreau, and C. Ringeissen. Handling ELAN Rewrite Programs via an Exchange Format. In C. Kirchner and H. Kirchner, editors, *Proc. Second Intl. Workshop on Rewriting Logic and its Applications*, Electronic Notes in Theoretical Computer Science, Pont-à-Mousson (France), September 1998. Elsevier.
- [BK97] P. Borovanský and H. Kirchner. Strategies of ELAN: meta-interpretation and partial evaluation. In *Proceedings of International Workshop on Theory and Practice of Algebraic Specifications ASF+SDF 97*, Workshops in Computing, Amsterdam, September 1997. Springer-Verlag.
- [BKK96a] P. Borovanský, C. Kirchner, and H. Kirchner. Controlling Rewriting by Rewriting. In J. Meseguer, editor, *Proc. First Intl. Workshop on Rewriting Logic and its Applications*, volume 4 of *Electronic Notes in Theoretical Computer Science*, Asilomar (California), September 1996. Elsevier.
- [BKK⁺96b] P. Borovanský, C. Kirchner, H. Kirchner, P.-E. Moreau, and M. Vittek. ELAN: A logical framework based on computational systems. In J. Meseguer, editor, *Proc. First Intl. Workshop on Rewriting Logic and its Applications*, volume 4 of *Electronic Notes in Theoretical Computer Science*, Asilomar (California), September 1996. Elsevier.
- [BKK97] P. Borovanský, C. Kirchner, and H. Kirchner. Rewriting as a Unified Specification Tool for Logic and Control: The ELAN Language. In *Proceedings of International Workshop on Theory and Practice of Algebraic Specifications ASF+SDF 97*, Workshops in Computing, Amsterdam, September 1997. Springer-Verlag.
- [BKK98a] P. Borovanský, C. Kirchner, and H. Kirchner. A functional view of rewriting and strategies for a semantics of ELAN. In M. Sato and Y. Toyama, editors, *The Third Fuji International Symposium on Functional and Logic Programming*, pages 143–167, Kyoto, April 1998. World Scientific.

- [BKK⁺98b] P. Borovanský, C. Kirchner, H. Kirchner, P.-E. Moreau, and C. Ringeissen. An Overview of ELAN. In C. Kirchner and H. Kirchner, editors, *Proc. Second Intl. Workshop on Rewriting Logic and its Applications*, Electronic Notes in Theoretical Computer Science, Pont-à-Mousson (France), September 1998. Elsevier.
- [Bor95] P. Borovanský. Implementation of higher-order unification based on calculus of explicit substitutions. In *Proceedings of SOFSEM'95*, Lecture Notes in Computer Science, pages 363–368. Springer-Verlag, 1995.
- [Bor98a] P. Borovanský. *Le contrôle de la réécriture : étude et implantation d'un formalisme de stratégies*. Thèse de Doctorat d'Université, Université Henri Poincaré - Nancy 1, octobre 1998.
- [Bor98b] P. Borovanský. The Control of Rewriting: Study and Implementation of a Strategy Formalism. In C. Kirchner and H. Kirchner, editors, *Proc. Second Intl. Workshop on Rewriting Logic and its Applications*, Electronic Notes in Theoretical Computer Science, Pont-à-Mousson (France), September 1998. Elsevier.
- [Cas96a] C. Castro. Binary CSP Solving as an Inference Process. In *Proceedings of the Eighth International Conference on Tools in Artificial Intelligence, ICTAI'96, Toulouse, France*, pages 462–463, November 1996.
- [Cas96b] C. Castro. Solving Binary CSP using Computational Systems. In J. Meseguer, editor, *Proc. First Intl. Workshop on Rewriting Logic and its Applications*, volume 4 of *Electronic Notes in Theoretical Computer Science*, Asilomar (California), September 1996. Elsevier.
- [Cas97a] C. Castro. Constraint Manipulation using Rewrite Rules and Strategies. In A. Drewery, G.-J. M. Kruijff, and R. Zuber, editors, *Proceedings of 2nd ESSLLI Student Session, 9th European Summer School in Logic, Language and Information, ESSLLI'97*, pages 45–56, Aix-en-Provence (France), August 1997.
- [Cas97b] C. Castro. Constraint Manipulation using Rewrite Rules and Strategies. In A. Drewery, G.-J. M. Kruijff, and R. Zuber, editors, *Proceedings of the Second ESSLLI Student Session, 9th European Summer School in Logic, Language and Information, ESSLLI'97*, pages 45–56, Aix-en-Provence, France, August 1997.
- [Cas98a] C. Castro. Building Constraint Satisfaction Problem Solvers Using Rewrite Rules and Strategies. *Fundamenta Informaticae*, 34(3):263–293, September 1998.
- [Cas98b] C. Castro. COLETTE, Prototyping CSP Solvers Using a Rule-Based Language. In *Proceedings of The Fourth International Conference on Artificial Intelligence and Symbolic Computation, Theory, Implementations and Applications, AISC'98*, volume 1476 of *Lecture Notes in Artificial Intelligence*, Plattsburgh, NY, USA, September 1998.
- [Cas98c] C. Castro. *Une approche déductive de la résolution de problèmes de satisfaction de contraintes*. Thèse de doctorat d'université, Université Henri Poincaré – Nancy 1, 1998. Also available as Technical Report 98-T-315, LORIA, Nancy (France).
- [Cir99] H. Cirstea. Specifying authentication protocols using ELAN. In *Workshop on Modelling and Verification*, Besancon, France, December 1999.

- [CK97] H. Cirstea and C. Kirchner. Theorem proving Using Computational Systems: The case of the B Predicate Prover. Available at <http://www.loria.fr/~cirstea/Papers/TheoremProver.ps>, 1997.
- [CK98] C. Castro and C. Kirchner. Constraint Rewriting. In *Proceedings of The Workshop on Applications of Rewriting, 9th International Conference on Rewriting Techniques and Applications, RTA'98, Tsukuba, Japan*, March 1998.
- [CK99a] H. Cirstea and C. Kirchner. Combining higher-order and first-order computation using ρ -calculus: Towards a semantics of ELAN. In D. Gabbay and M. de Rijke, editors, *Frontiers of Combining Systems 2*, Research Studies, ISBN 0863802524, pages 95–120. Wiley, 1999.
- [CK99b] H. Cirstea and C. Kirchner. An introduction to the rewriting calculus. Research Report RR-3818, INRIA, December 1999.
- [DK98] H. Dubois and H. Kirchner. Actions and plans in ELAN. In *Proceedings of the Workshop on Strategies in Automated Deduction - CADE-15, Lindau, Germany*, pages 35–45, 1998.
- [DK99] H. Dubois and H. Kirchner. Rule based programming with constraints and strategies. Technical Report 99-R-084, LORIA, Nancy, France, November 1999. ERCIM workshop on Constraints, Paphos (Cyprus).
- [Ear70] J. Earley. An efficient context-free parsing algorithm. *Communications of the ACM*, 13(2):94–102, 1970.
- [Eke95] S. Eker. Associative-commutative matching via bipartite graph matching. *Computer Journal*, 38(5):381–399, 1995.
- [Gen98a] T. Genet. *Contraintes d'ordre et automates d'arbres pour les preuves de terminaison*. Thèse de Doctorat d'Université, Université Henri Poincaré – Nancy 1, 1998.
- [Gen98b] T. Genet. Decidable Approximations of Sets of Descendants and Sets of Normal Forms. In *Proceedings 9th Conference on Rewriting Techniques and Applications, Tsukuba (Japan)*, volume 1379 of *Lecture Notes in Computer Science*, pages 151–165. Springer-Verlag, 1998.
- [GG97] T. Genet and I. Gnaedig. Termination Proofs using gpo Ordering Constraints. In M. Bidoit and M. Dauchet, editors, *Proceedings 22nd International Colloquium on Trees in Algebra and Programming, Lille (France)*, volume 1214 of *Lecture Notes in Computer Science*, pages 249–260. Springer-Verlag, 1997.
- [GKK⁺87] J. A. Goguen, C. Kirchner, H. Kirchner, A. Mégreli, J. Meseguer, and T. Winkler. An introduction to OBJ-3. In J.-P. Jouannaud and S. Kaplan, editors, *Proceedings 1st International Workshop on Conditional Term Rewriting Systems, Orsay (France)*, volume 308 of *Lecture Notes in Computer Science*, pages 258–263. Springer-Verlag, July 1987. Also as internal report CRIN: 88-R-001.
- [HHKR89] J. Heering, P. Hendriks, P. Klint, and J. Rekers. The syntax definition formalisme SDF - reference manual. *ACM SIGPLAN Notices*, 24(11):43–75, 1989.
- [Kir97] H. Kirchner. ELAN (tutoriel invité). In F. Fages, editor, *JFPLC'99 Journées Francophones de Programmation Logique et programmation par Contraintes*, pages 241–248. Hermes Science Publications, 1997. Report LORIA 99-R-129.

- [KKV95] C. Kirchner, H. Kirchner, and M. Vittek. Designing Constraint Logic Programming Languages using Computational Systems. In P. Van Hentenryck and V. Saraswat, editors, *Principles and Practice of Constraint Programming. The Newport Papers.*, pages 131–158. MIT press, 1995.
- [KM95] H. Kirchner and P.-E. Moreau. Prototyping completion with constraints using computational systems. In J. Hsiang, editor, *Proceedings 6th Conference on Rewriting Techniques and Applications, Kaiserslautern (Germany)*, volume 914 of *Lecture Notes in Computer Science*, pages 438–443. Springer-Verlag, 1995.
- [KM96] H. Kirchner and P.-E. Moreau. A reflective extension of ELAN. In J. Meseguer, editor, *Proc. First Intl. Workshop on Rewriting Logic and its Applications*, volume 4 of *Electronic Notes in Theoretical Computer Science*, Asilomar (California), September 1996. Elsevier.
- [KM98] H. Kirchner and P.-E. Moreau. Non-deterministic Computations in ELAN, 1998. Submitted.
- [KR98] C. Kirchner and C. Ringeissen. Rule-Based Constraint Programming. *Fundamenta Informaticae*, 34(3):225–262, September 1998.
- [Mes92] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.
- [MK97] P.-E. Moreau and H. Kirchner. Compilation Techniques for Associative-Commutative Normalisation. In *Proceedings of International Workshop on Theory and Practice of Algebraic Specifications ASF+SDF 97*, Workshops in Computing, Amsterdam, September 1997. Springer-Verlag.
- [MK98] P. Moreau and H. Kirchner. A compiler for rewrite programs in associative-commutative theories. In C. Palamidessi, H. Glaser, and K. Meinke, editors, *Principles of Declarative Programming*, number 1490 in *Lecture Notes in Computer Science*, pages 230–249. Springer-Verlag, September 1998.
- [Mor98] P.-E. Moreau. A choice-point library for backtrack programming. JICSLP’98 Post-Conference Workshop on Implementation Technologies for Programming Languages based on Logic, June 1998.
- [Mor99] P.-E. Moreau. *Compilation de règles de réécriture et de stratégies non-déterministes*. Thèse de Doctorat d’Université, Université Henri Poincaré – Nancy 1, France, June 1999. also TR LORIA 98-T-326.
- [Ngu] Q.-H. Nguyen. ELAN for AC rewriting in Coq. <http://www.loria.fr/~nguyenqh/coqelan/ACrew.html>.
- [Rin97] C. Ringeissen. Prototyping Combination of Unification Algorithms with the ELAN Rule-Based Programming Language. In *Proceedings 8th Conference on Rewriting Techniques and Applications, Sitges (Spain)*, volume 1232 of *Lecture Notes in Computer Science*, pages 323–326. Springer-Verlag, 1997.
- [Vir96] P. Viry. Input/Output for ELAN. In J. Meseguer, editor, *Proc. First Intl. Workshop on Rewriting Logic and its Applications*, volume 4 of *Electronic Notes in Theoretical Computer Science*, Asilomar (California), September 1996. Elsevier.

- [Vit94] M. Vittek. *ELAN: Un cadre logique pour le prototypage de langages de programmation avec contraintes*. Thèse de Doctorat d'Université, Université Henri Poincaré - Nancy 1, octobre 1994.
- [Vit96] M. Vittek. A Compiler for Nondeterministic Term Rewriting Systems. In H. Ganzinger, editor, *Proceedings 7th Conference on Rewriting Techniques and Applications, New Brunswick (New Jersey, USA)*, volume 1103 of *Lecture Notes in Computer Science*, pages 154–168. Springer-Verlag, July 1996.